Bachelor Mathematics

Bachelor thesis

Scalable Gaussian Process Regression through State-Space Formulations

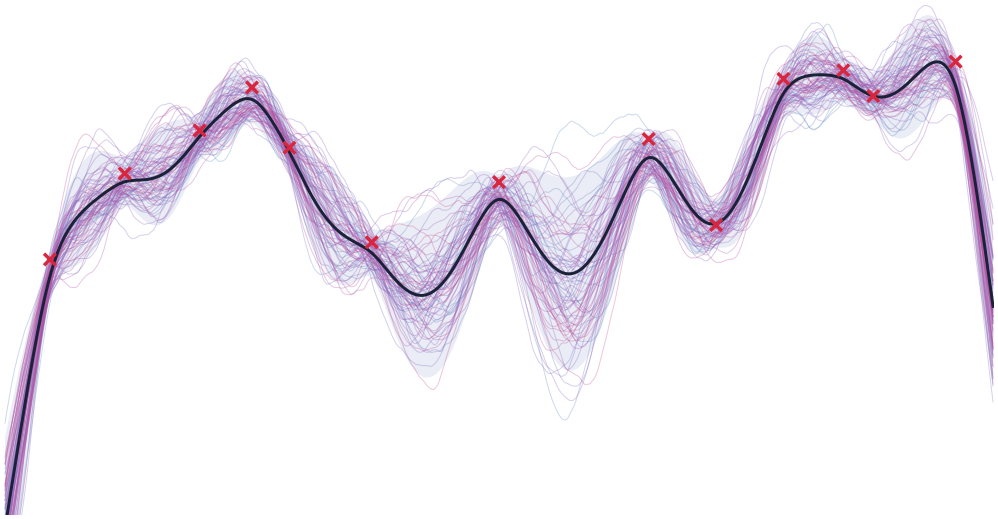
by

Andreas Alexandrou

June 30, 2026

Supervisor: prof. dr. ir. Frank van der Meulen

Second examiner: dr. Oliver Fabert



Department of Mathematics
Faculty of Science

VU VRIJE UNIVERSITEIT AMSTERDAM

Abstract

Gaussian process regression is a flexible nonparametric method for inferring an unknown function from noisy data, but it scales poorly: the posterior requires inverting a dense $n \times n$ covariance matrix, at a cost that grows cubically in the number of observations n. In this thesis we show that this cubic cost can be avoided for an important class of kernels. For a stationary, temporal (one-dimensional) Gaussian process whose spectral density is rational, the prior can be realized as the output of a linear time-invariant stochastic differential equation. Discretizing this equation at the observation times yields a linear-Gaussian state-space model, to which the Kalman filter, the closed-form solution of the Bayesian filtering equations, applies directly. A subsequent backward pass with the Rauch–Tung–Striebel smoother conditions every state on the full dataset and recovers exactly the Gaussian process regression posterior, now at a cost that grows only linearly in n. We develop the full construction from first principles and carry it out explicitly for the Matérn family of covariance functions. A direct implementation reproduces dense Gaussian process regression to machine precision while confirming the predicted linear scaling.

Title: Scalable Gaussian Process Regression through State-Space Formulations
Author: Andreas Alexandrou, a.alexandrou2@student.vu.nl, 2802493
Supervisor: prof. dr. ir. Frank van der Meulen
Second examiner: dr. Oliver Fabert
Date: June 30, 2026

Department of Mathematics
Vrije Universiteit Amsterdam
de Boelelaan 1081, 1081 HV Amsterdam
`http://www.math.vu.nl/`

Contents

1 Introduction **4**

2 Background **7**
2.1 Gaussian Distributions and Covariance Functions 7
2.2 Brownian Motion and White Noise 8
2.3 Spectral and Stochastic-Calculus Tools 9

3 Gaussian Process Regression **11**
3.1 Gaussian Processes 11
3.2 Gaussian Process Regression 12

4 State-Space Formulation of Temporal Gaussian Processes **15**
4.1 The LTI SDE Representation 15
4.2 Spectral Density and Covariance Function of the Output Process 16
4.3 Rationality of the Spectral Density 19
4.4 Constructing an LTI SDE from a Covariance Function 20

5 State-Space Inference **22**
5.1 Discretization of the Continuous-Time State-Space Model 22
5.2 Bayesian Filtering Equations 24
5.3 The Kalman Filter 25
5.4 Bayesian Smoothing Equations 26
5.5 Rauch–Tung–Striebel Smoother 27
5.6 The Computational Advantage 28
5.7 Hyperparameter Learning 28

6 The Construction Recipe and the Matérn Covariance Family **30**
6.1 The Construction Recipe 30
6.2 The Matérn Covariance Family 31

7 Experiments **34**
7.1 Experimental Setup 34
7.2 Posterior Equivalence 35
7.3 Computational Cost and Scaling 36

8 Conclusion **38**

1 Introduction

Regression is the problem of inferring an unknown function from noisy observations. It arises throughout the quantitative sciences, for example in time-series analysis, signal processing, sensor data, and geostatistics. The classical *parametric* approach fixes the functional form of the relationship in advance: linear regression, for example, assumes that the output depends linearly on the input, and estimates a fixed number of parameters regardless of how much data is available. This works well when the form of the underlying function is known, but is restrictive when it is not. *Nonparametric* regression removes this assumption and lets the data, rather than a fixed template, determine the shape of the function. Gaussian process (GP) regression realizes this idea in a fully probabilistic way: rather than parameterizing the function, it places a prior distribution directly over functions. Given observations, which we model as function values corrupted by independent Gaussian noise, it returns a posterior that is again Gaussian, with an explicit mean and variance: the mean serves as the prediction, and the variance quantifies the remaining uncertainty.

This combination of flexibility and built-in uncertainty quantification has made GP regression a standard tool in machine learning and spatial statistics [14]. It comes, however, at a price. Exact GP regression requires inverting the dense $n \times n$ covariance matrix of the observations, where n is the number of data points. This inversion takes $\mathcal{O}(n^3)$ operations, while merely storing the matrix takes $\mathcal{O}(n^2)$ memory. A range of techniques reduce this cost, usually by exploiting special structure in the covariance matrix, but in general the cubic scaling is the central bottleneck that keeps GP regression from the large datasets now routine in time-series and sensor applications. The question this thesis addresses is whether the *exact* GP posterior, not an approximation to it, can be recovered at a cost that is *linear*, rather than cubic, in n. We show that this is possible for an important class of priors: stationary processes on a single (temporal) input whose spectral density is a rational function of the squared angular frequency ω^2.

Before describing our route, we place it among existing approaches. The Matérn covariance function, which will serve as our central example, dates back to Matérn's 1960 work on spatial statistics [12], and GP regression was brought into the machine-learning mainstream by Rasmussen and Williams [14]. The cubic-cost problem has since been addressed from several directions. One line of work sacrifices exactness for speed through sparse or inducing-point approximations [13, 20]. A complementary line keeps the result exact but accelerates the linear algebra by exploiting structure in the covariance matrix, from the Cholesky factorization underlying the standard dense solver [4, 14] to fast solvers for Toeplitz, Kronecker, or hierarchical matrices [21, 1], though these apply only to covariance matrices of the corresponding special form. The approach taken here belongs instead to a tradition with roots in classical filtering theory, going back to

the recursive estimator of Kalman [8] and the smoothing recursion of Rauch, Tung and Striebel [15], and developed for Gaussian processes in the state-space framework of Hartikainen and Särkkä [7] and the textbook of Särkkä and Solin [16].

The idea behind this approach is as follows. Its starting point is a linear time-invariant (LTI) stochastic differential equation (SDE), that is, a differential equation driven by white noise. Once discretized at the observation times, such an equation becomes a linear-Gaussian state-space model, for which the Bayesian filtering equations have a closed-form solution: the Kalman filter, which processes the observations sequentially. A subsequent backward pass, the Rauch–Tung–Striebel (RTS) smoother, conditions each state on the entire dataset and yields the posterior. Hartikainen and Särkkä [7] showed that a temporal GP prior can itself be written as such an LTI SDE, so that this forward and backward procedure returns exactly the GP regression posterior, at a cost linear in n. The condition that makes this possible, and on which the whole thesis rests, is that the GP prior have a rational spectral density which is precisely what allows the process to be represented by a finite-dimensional SDE. Under this condition the batch matrix algebra of GP regression is replaced by sequential filtering and smoothing, reducing the cost from $\mathcal{O}(n^3)$ to $\mathcal{O}(nm^3)$, where the state dimension m is fixed and small; the construction is exact, the only departure from the dense result being that of floating-point arithmetic.

The aim of this thesis is to give a self-contained, rigorous, and student-readable derivation of this state-space formulation of temporal GP regression, developing every step from first principles and carrying it out explicitly for the Matérn family, in full detail for the Matérn-3/2 case used in the experiments. We then validate the construction empirically, confirming that it reproduces the dense GP posterior up to numerical error and that its running time scales linearly in n. Throughout we restrict attention to a temporal input, namely a single ordered real variable t (the ordering being what the sequential filter relies on), and to stationary covariance functions with rational spectral density; for the Matérn family this corresponds to half-integer smoothness. The kernel hyperparameters are treated as fixed and given, though we show that the same forward pass used for inference also yields the marginal likelihood of the data at no extra cost, so that they can in principle be learned within the same framework. These restrictions are exactly what make the exact finite-dimensional representation possible.

With the goal set, we outline the path. Chapter 2 collects the probabilistic, Fourier-analytic, and stochastic-calculus definitions used throughout. Chapter 3 introduces Gaussian processes and derives the GP regression equations, making the cubic bottleneck explicit. Chapter 4 develops the LTI SDE representation, shows that the spectral density of its output is rational in ω^2, and then runs the argument in reverse to construct an SDE from a given stationary covariance via spectral factorization. Chapter 5 discretizes the model and derives Kalman filtering and RTS smoothing as the inference engine. Chapter 6 distills the construction into a step-by-step recipe and applies it to the Matérn family. Chapter 7 reports the numerical experiments, and Chapter 8 concludes with a discussion of limitations and an outlook on future work, including the spatio-temporal setting.

A word on the nature of this contribution. The state-space view of GP regression is well established; the spectral density of an LTI SDE output, for instance, is a standard

result [16], and this thesis contains no new theorem. Its aim is expository. The literature on the subject is dense and largely written for readers already fluent in stochastic differential equations and filtering, and much of it is engineering-oriented, stating results rather than proving them. Here we gather the construction into one place, written for a fellow mathematics student, and supply full proofs of each step. The one point where we go beyond reorganizing known material is in Chapter 4, where the output spectral density is derived elementarily and directly from the companion-form matrices.

2 Background

In this chapter, we collect the main definitions and notation used throughout the thesis.

Notation. Throughout this thesis, bold uppercase symbols denote matrices and bold lowercase symbols denote vectors. Scalars are written in ordinary font.

2.1 Gaussian Distributions and Covariance Functions

We begin with the multivariate Gaussian distribution and the conditioning formula that, as we will see in Chapter 3, is the single computational engine behind Gaussian process regression.

Definition 2.1 (Multivariate Gaussian distribution)**.** A random vector $\mathbf{z} \in \mathbb{R}^n$ is said to follow a multivariate Gaussian distribution, written

$$\mathbf{z} \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma}),$$

if it has density

$$p(\mathbf{z}) = \frac{1}{(2\pi)^{n/2}|\boldsymbol{\Sigma}|^{1/2}} \exp\left(-\frac{1}{2}(\mathbf{z}-\boldsymbol{\mu})^\top \boldsymbol{\Sigma}^{-1}(\mathbf{z}-\boldsymbol{\mu})\right),$$

where $\boldsymbol{\mu} \in \mathbb{R}^n$ is the mean vector and $\boldsymbol{\Sigma} \in \mathbb{R}^{n\times n}$ is a symmetric positive definite covariance matrix. Any subvector of a Gaussian vector is again Gaussian [3].

The following standard fact, that conditioning one part of a jointly Gaussian vector on another again yields a Gaussian, with mean and covariance available in closed form, is what makes the Gaussian process posterior explicit.

Lemma 2.2 (Conditioning of a jointly Gaussian vector)**.** *Let* $\mathbf{a} \in \mathbb{R}^{d_a}$ *and* $\mathbf{b} \in \mathbb{R}^{d_b}$ *be jointly Gaussian,*

$$\begin{pmatrix}\mathbf{a}\\ \mathbf{b}\end{pmatrix} \sim \mathcal{N}\left(\begin{pmatrix}\boldsymbol{\mu}_a\\ \boldsymbol{\mu}_b\end{pmatrix}, \begin{pmatrix}\boldsymbol{\Sigma}_{aa} & \boldsymbol{\Sigma}_{ab}\\ \boldsymbol{\Sigma}_{ba} & \boldsymbol{\Sigma}_{bb}\end{pmatrix}\right)$$

with $\boldsymbol{\Sigma}_{bb}$ *invertible. Then* $\mathbf{a} \mid \mathbf{b}$ *is Gaussian [14],*

$$\mathbf{a} \mid \mathbf{b} \sim \mathcal{N}\big(\boldsymbol{\mu}_a + \boldsymbol{\Sigma}_{ab}\boldsymbol{\Sigma}_{bb}^{-1}(\mathbf{b}-\boldsymbol{\mu}_b),\ \boldsymbol{\Sigma}_{aa} - \boldsymbol{\Sigma}_{ab}\boldsymbol{\Sigma}_{bb}^{-1}\boldsymbol{\Sigma}_{ba}\big).$$

A Gaussian process is specified through its covariance function, which we define next.

Definition 2.3 (Covariance function / kernel)**.** Let $f(t)$ be a random process. Its covariance function is defined by

$$k(t,s) = \mathrm{Cov}(f(t), f(s)) = \mathbb{E}[(f(t) - \mathbb{E}[f(t)])(f(s) - \mathbb{E}[f(s)])] \,.$$

The function k is symmetric, and for any $t_1, \ldots, t_n \in \mathbb{R}$ and $c_1, \ldots, c_n \in \mathbb{R}$ it satisfies

$$\sum_{i=1}^{n} \sum_{j=1}^{n} c_i c_j k(t_i, t_j) \geq 0.$$

In the Gaussian process literature, the covariance function is also called the kernel [14, 5].

The covariance functions treated in this thesis are stationary, depending on the two inputs only through their difference. This property is what will eventually allow the corresponding process to be represented by a time-invariant state-space model.

Definition 2.4 (Stationary covariance function)**.** A covariance function is called stationary if it depends only on the difference $\tau = t - s$, that is, there exists a $\bar{k}$ such that [16]

$$k(t,s) = \bar{k}(t-s) = \bar{k}(\tau).$$

Remark. For notational convenience, in the remainder of the thesis we write $k(\tau)$ for $\bar{k}(\tau)$ whenever the covariance function is stationary, suppressing the bar.

2.2 Brownian Motion and White Noise

The state-space models of the later chapters are driven by white noise, the formal derivative of Brownian motion. We recall both here, together with the intensity convention we adopt throughout.

Definition 2.5 (Brownian motion)**.** A standard Brownian motion $\beta(t)$, $t \geq 0$, is a stochastic process satisfying the following properties:

- $\beta(0) = 0$.
- **Independent increments:** for $t > s$, the increment $\beta(t) - \beta(s)$ is independent of the past.
- **Gaussian increments:** for $t > s$,

$$\beta(t) - \beta(s) \sim \mathcal{N}(0, t - s).$$

- **Continuous sample paths:** the paths $t \mapsto \beta(t)$ are continuous.

Remark. In the LTI SDE formulation used later, the driving Brownian motion is allowed to have intensity $q > 0$. This means that increments over an interval of length $t - s$ have variance $q(t - s)$. Equivalently, this can be viewed as multiplying a standard Brownian motion by $\sqrt{q}$. We use this convention because the corresponding driving white noise then has spectral density q, matching the notation used in the state-space Gaussian process literature [16, 9].

Definition 2.6 (White noise)**.** White noise $w(t)$ may be viewed formally as the derivative of Brownian motion,

$$w(t) = \frac{d\beta(t)}{dt},$$

in the sense of distributions. With the intensity convention above, it is characterized formally by

$$\mathbb{E}[w(t)] = 0, \qquad \mathbb{E}[w(t)w(t')] = q\,\delta(t - t'),$$

where δ denotes the Dirac delta distribution. Thus, white noise is not defined pointwise in the classical sense and is meaningful only under an integral [16, 9].

2.3 Spectral and Stochastic-Calculus Tools

Finally, we collect the Fourier transform and spectral density, which relate a stationary covariance function to its frequency content, together with two Itô identities needed for the covariance computations of Chapter 4.

Definition 2.7 (Fourier transform)**.** Let $f : \mathbb{R} \to \mathbb{R}$ be an integrable function. Its Fourier transform is the complex-valued function $\hat{f} : \mathbb{R} \to \mathbb{C}$ defined by

$$\hat{f}(\omega) = \mathcal{F}[f](\omega) = \int_{-\infty}^{\infty} f(t)e^{-i\omega t}\,dt,$$

with inverse transform

$$f(t) = \mathcal{F}^{-1}[\hat{f}](t) = \frac{1}{2\pi}\int_{-\infty}^{\infty} \hat{f}(\omega)e^{i\omega t}\,d\omega.$$

A property used repeatedly below is

$$\mathcal{F}\left[\frac{d^n f}{dt^n}\right](\omega) = (i\omega)^n \hat{f}(\omega), \qquad n \geq 1.$$

Lemma 2.8 (Itô product rule)**.** *Let $\mathbf{x}(t)$ satisfy the Itô SDE $d\mathbf{x} = \mathbf{a}\,dt + \mathbf{L}\,d\beta(t)$ with scalar Brownian motion of intensity q [9]. Then*

$$d\big(\mathbf{x}\mathbf{x}^\top\big) = (d\mathbf{x})\,\mathbf{x}^\top + \mathbf{x}\,(d\mathbf{x})^\top + (d\mathbf{x})(d\mathbf{x})^\top,$$

where the quadratic term is evaluated with the Itô rules $dt\,dt = dt\,d\beta = 0$ and $d\beta\,d\beta = q\,dt$.

Lemma 2.9 (Itô isometry)**.** *For a deterministic integrand* $\mathbf{G}(s)$ *and scalar Brownian motion of intensity* q *[9],*

$$\mathbb{E}\left[\left(\int_0^T \mathbf{G}(s)\, d\beta(s)\right)\left(\int_0^T \mathbf{G}(s)\, d\beta(s)\right)^\top\right] = q\int_0^T \mathbf{G}(s)\,\mathbf{G}(s)^\top\, ds.$$

Definition 2.10 (Spectral density)**.** For a stationary process $X(t)$, the spectral density is the frequency-domain description of its covariance structure. In the LTI SDE setting, it is formally written as

$$S_X(\omega) = \mathbb{E}\left[|\hat{X}(\omega)|^2\right] = \mathbb{E}\left[\hat{X}(\omega)\hat{X}(-\omega)\right].$$

For a vector-valued process $\mathbf{x}(t)$, this becomes the spectral density matrix

$$\mathbf{S}_{\mathbf{x}}(\omega) = \mathbb{E}\left[\hat{\mathbf{x}}(\omega)\hat{\mathbf{x}}(-\omega)^\top\right].$$

The covariance function for the scalar-valued process is recovered by the inverse Fourier transform

$$k_X(\tau) = \frac{1}{2\pi}\int_{-\infty}^{\infty} S_X(\omega)e^{i\omega\tau}\, d\omega.$$

This Fourier-domain notation is formal for white noise, but is standard in the LTI SDE setting used later [16].

With these definitions in place, we turn to Gaussian processes themselves.

3 Gaussian Process Regression

The Gaussian process extends the finite-dimensional Gaussian distributions of Chapter 2 from random vectors to random functions, realizing the nonparametric idea sketched in the introduction: rather than committing to a parametric family in advance, one places a prior distribution directly over functions, so that the effective complexity of the model is not fixed beforehand but grows with the data. In this chapter we introduce Gaussian processes, derive the standard Gaussian process regression equations, and then identify the computational bottleneck of direct Gaussian process regression that motivates the state-space formulation of the following chapters.

3.1 Gaussian Processes

Instead of placing a Gaussian distribution on a finite-dimensional parameter vector, a Gaussian process places a Gaussian distribution on any finite collection of function values.

Definition 3.1 (Gaussian process)**.** Let $\mathbf{t} \in \mathbb{R}^D$ where $D \in \mathbb{N}$. A Gaussian process (GP) is a random function $f(\mathbf{t})$ such that for any finite collection of input points $\mathbf{t}_1, \mathbf{t}_2, \ldots, \mathbf{t}_n \in \mathbb{R}^D$, the random vector

$$\big(f(\mathbf{t}_1), f(\mathbf{t}_2), \ldots, f(\mathbf{t}_n)\big)^\top$$

has a multivariate Gaussian distribution.

A Gaussian process is completely characterized by its mean function

$$m(\mathbf{t}) = \mathbb{E}[f(\mathbf{t})]$$

and covariance function

$$k(\mathbf{t}, \mathbf{t}') = \mathbb{E}\left[(f(\mathbf{t}) - m(\mathbf{t}))(f(\mathbf{t}') - m(\mathbf{t}'))\right].$$

We write

$$f(\mathbf{t}) \sim \mathcal{GP}(m(\mathbf{t}), k(\mathbf{t}, \mathbf{t}')).$$

This notation means that for every finite set of input points $\mathbf{t}_1, \ldots, \mathbf{t}_n \in \mathbb{R}^D$ [14, 18],

$$\begin{pmatrix} f(\mathbf{t}_1) \\ \vdots \\ f(\mathbf{t}_n) \end{pmatrix} \sim \mathcal{N}\left(\begin{pmatrix} m(\mathbf{t}_1) \\ \vdots \\ m(\mathbf{t}_n) \end{pmatrix}, \begin{pmatrix} k(\mathbf{t}_1, \mathbf{t}_1) & \cdots & k(\mathbf{t}_1, \mathbf{t}_n) \\ \vdots & \ddots & \vdots \\ k(\mathbf{t}_n, \mathbf{t}_1) & \cdots & k(\mathbf{t}_n, \mathbf{t}_n) \end{pmatrix}\right).$$

3.2 Gaussian Process Regression

Gaussian process regression concerns inferring the posterior distribution of an unknown function

$$f : \mathbb{R}^D \to \mathbb{R},$$

at an arbitrary input $\mathbf{t}_* \in \mathbb{R}^D$, given a dataset

$$\mathcal{D} = \{(\mathbf{t}_i, y_i)\}_{i=1}^n,$$

where $y_i \in \mathbb{R}$ are the observed outputs corresponding to the inputs $\mathbf{t}_i$. A common observation model is

$$y_i = f(\mathbf{t}_i) + \varepsilon_i, \qquad \varepsilon_i \overset{\text{i.i.d.}}{\sim} \mathcal{N}(0, \sigma^2_{\text{noise}}).$$

We assume that the noise variables ε_i are independent of the Gaussian process f. This is the standard Gaussian observation model used in Gaussian process regression [14, 7]. Now let

$$\mathbf{y} = \begin{pmatrix} y_1 \\ \vdots \\ y_n \end{pmatrix}, \qquad \mathbf{f} = \begin{pmatrix} f(\mathbf{t}_1) \\ \vdots \\ f(\mathbf{t}_n) \end{pmatrix}, \qquad \boldsymbol{\varepsilon} = \begin{pmatrix} \varepsilon_1 \\ \vdots \\ \varepsilon_n \end{pmatrix} \sim \mathcal{N}\big(\mathbf{0}, \sigma^2_{\text{noise}}\mathbf{I}_n\big).$$

The observation model can therefore be written in vector form as

$$\mathbf{y} = \mathbf{f} + \boldsymbol{\varepsilon}.$$

In Gaussian process regression, we assume the prior

$$f \sim \mathcal{GP}(0, k(\mathbf{t}, \mathbf{t}')).$$

It follows that

$$\mathbf{f} \sim \mathcal{N}(\mathbf{0}, \mathbf{K}),$$

where

$$[\mathbf{K}]_{ij} = k(\mathbf{t}_i, \mathbf{t}_j).$$

Now let

$$f^* := f(\mathbf{t}_*),$$

where $\mathbf{t}_*$ is an arbitrary unseen input. We refer to $\mathbf{f}$ as the vector of training function values and to f^* as the test function value. By the properties of Gaussian processes, the training function values and the test function value are jointly Gaussian

$$\begin{pmatrix} \mathbf{f} \\ f^* \end{pmatrix} \sim \mathcal{N}\left(\begin{pmatrix} \mathbf{0} \\ 0 \end{pmatrix}, \begin{pmatrix} \mathbf{K} & k_{*,\mathbf{f}}^\top \\ k_{*,\mathbf{f}} & k(\mathbf{t}_*, \mathbf{t}_*) \end{pmatrix}\right),$$

where $k_{*,\mathbf{f}} \in \mathbb{R}^{1\times n}$ is the row vector

$$k_{*,\mathbf{f}} = \begin{pmatrix} k(\mathbf{t}_*, \mathbf{t}_1) & \cdots & k(\mathbf{t}_*, \mathbf{t}_n) \end{pmatrix}.$$

We next derive the joint distribution of the vector

$$\begin{pmatrix} \mathbf{y} \\ f^* \end{pmatrix}.$$

Since

$$\mathbf{y} = \mathbf{f} + \boldsymbol{\varepsilon},$$

and since $\mathbf{f}$, f^* and $\boldsymbol{\varepsilon}$ are jointly Gaussian with $\boldsymbol{\varepsilon}$ independent of the Gaussian process f, the vector

$$\begin{pmatrix} \mathbf{y} \\ f^* \end{pmatrix}$$

is also jointly Gaussian. Therefore, it remains to compute its mean vector and covariance matrix. Since both $\mathbf{f}$ and $\boldsymbol{\varepsilon}$ have zero mean, we have

$$\mathbb{E}[\mathbf{y}] = \mathbb{E}[\mathbf{f} + \boldsymbol{\varepsilon}] = \mathbf{0}.$$

Also,

$$\mathbb{E}[f^*] = 0,$$

because the GP prior has zero mean. For the covariance matrix of the observation vector, we obtain

$$\mathrm{Cov}(\mathbf{y}) = \mathrm{Cov}(\mathbf{f} + \boldsymbol{\varepsilon}).$$

By the law of total covariance we have

$$\mathrm{Cov}(\mathbf{y}) = \mathrm{Cov}(\mathbb{E}[\mathbf{y} \mid \mathbf{f}]) + \mathbb{E}[\mathrm{Cov}(\mathbf{y} \mid \mathbf{f})]$$

By the independence assumption and the fact that $\boldsymbol{\varepsilon}$ has zero mean, we have

$$\mathbb{E}[\mathbf{y} \mid \mathbf{f}] = \mathbf{f}, \qquad \mathrm{Cov}(\mathbf{y} \mid \mathbf{f}) = \sigma^2_{\text{noise}}\mathbf{I}_n.$$

Substituting gives

$$\mathrm{Cov}(\mathbf{y}) = \mathrm{Cov}(\mathbf{f}) + \mathbb{E}[\sigma^2_{\text{noise}}\mathbf{I}_n] = \mathbf{K} + \sigma^2_{\text{noise}}\mathbf{I}_n.$$

For the cross-covariance between the test function value and the observations,

$$\mathrm{Cov}(f^*, \mathbf{y}) = \mathrm{Cov}(f^*, \mathbf{f} + \boldsymbol{\varepsilon}).$$

By the same independence assumption,

$$\mathrm{Cov}(f^*, \boldsymbol{\varepsilon}) = 0.$$

Hence,

$$\mathrm{Cov}(f^*, \mathbf{y}) = \mathrm{Cov}(f^*, \mathbf{f}) = k_{*,\mathbf{f}}.$$

Therefore,

$$\begin{pmatrix} \mathbf{y} \\ f^* \end{pmatrix} \sim \mathcal{N}\left(\begin{pmatrix} \mathbf{0} \\ 0 \end{pmatrix}, \begin{pmatrix} \mathbf{K} + \sigma^2_{\text{noise}}\mathbf{I}_n & k_{*,\mathbf{f}}^\top \\ k_{*,\mathbf{f}} & k(\mathbf{t}_*, \mathbf{t}_*) \end{pmatrix} \right).$$

Then, by the conditioning formula for jointly Gaussian vectors (Lemma 2.2), the test function value conditioned on the observations is

$$f^* \mid \mathbf{y} \sim \mathcal{N}\left(\mu_{\mathcal{GP}}(\mathbf{t}_*), \sigma^2_{\mathcal{GP}}(\mathbf{t}_*)\right),$$

where

$$\mu_{\mathcal{GP}}(\mathbf{t}_*) = k_{*,\mathbf{f}}(\mathbf{K} + \sigma^2_{\text{noise}}\mathbf{I}_n)^{-1}\mathbf{y},$$
$$\sigma^2_{\mathcal{GP}}(\mathbf{t}_*) = k(\mathbf{t}_*, \mathbf{t}_*) - k_{*,\mathbf{f}}(\mathbf{K} + \sigma^2_{\text{noise}}\mathbf{I}_n)^{-1}k^\top_{*,\mathbf{f}}.$$

The posterior mean $\mu_{\mathcal{GP}}(\mathbf{t}_*)$ is a weighted combination of the observations, with weights determined by the covariance function. The posterior variance $\sigma^2_{\mathcal{GP}}(\mathbf{t}_*)$ decreases near observed inputs, reflecting reduced uncertainty, and reverts to the prior variance $k(\mathbf{t}_*, \mathbf{t}_*)$ far from any data. Figure 3.1 illustrates a typical posterior for the Matérn covariance function, which is discussed extensively in Chapter 6.

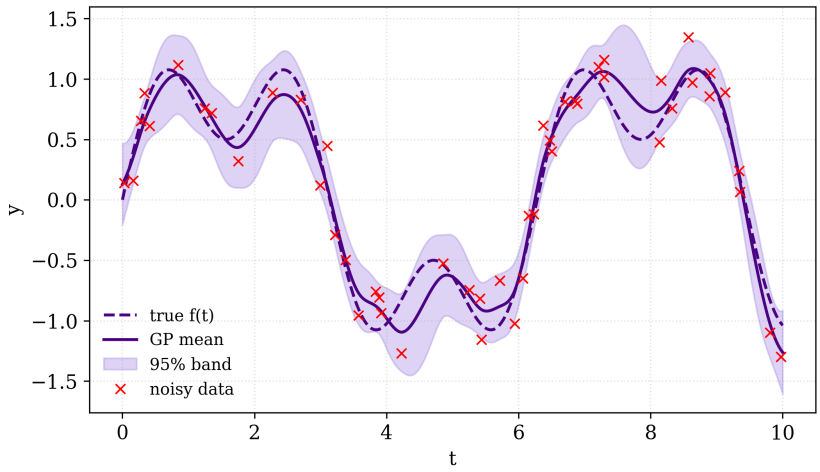


Figure 3.1: Gaussian process regression posterior for a Matérn $\nu = 3/2$ prior with length-scale $\ell = 1.06$, signal variance $\sigma^2 = 0.78$, and noise variance $\sigma^2_{\text{noise}} = 0.04$.

The main computational difficulty in Gaussian process regression consists of inverting the dense $n \times n$ matrix

$$\mathbf{K} + \sigma^2_{\text{noise}}\mathbf{I}_n.$$

For a general $n \times n$ covariance matrix, inversion requires $\mathcal{O}(n^3)$ computational cost and $\mathcal{O}(n^2)$ memory, where n is the number of observations. This is the computational bottleneck that motivates the state-space formulation used in the following chapters.

In the remainder of this thesis, we restrict attention to the temporal case $D = 1$, so that the input is a single scalar t, and to stationary covariance functions, for which we write $k(\tau)$ with $\tau = t - t'$ as in Chapter 2 [7].

4 State-Space Formulation of Temporal Gaussian Processes

We now turn to the state-space formulation of temporal Gaussian processes. The main idea is to represent a Gaussian process prior $f(t)$ as the output of a linear time-invariant (LTI) stochastic differential equation (SDE). When such a representation exists, the Gaussian process regression problem can be rewritten as inference in a linear Gaussian state-space model. The posterior distribution can then be computed sequentially by Kalman filtering and Rauch–Tung–Striebel smoothing, while preserving the same prior covariance structure.

The key question is therefore the following: given a stationary covariance function $k(\tau)$, can we construct an LTI SDE whose output process has covariance function $k(\tau)$? In general, this is possible for covariance functions whose spectral density admits a suitable rational representation. This includes the Matérn class used later in this thesis.

In this chapter, we lay the groundwork for answering this question. We first assume that we have been given an LTI SDE and find its spectral density $S_f(\omega)$ and the covariance function $k_f(\tau)$ of its output. Then we show that this spectral density $S_f(\omega)$ has a rational form. Finally, we run this argument in reverse: given a covariance function whose spectral density is rational in ω^2 we show how one can construct an LTI SDE via spectral factorization.

4.1 The LTI SDE Representation

We wish to represent the temporal process $f(t)$, with covariance function $k(\tau)$, as the output of a linear time-invariant stochastic differential equation. This is a special case of a continuous-time state-space model. Formally, we consider the white-noise driven equation [7, 16]

$$\frac{d\mathbf{x}(t)}{dt} = \mathbf{F}\mathbf{x}(t) + \mathbf{L}w(t), \tag{4.1}$$

where $w(t)$ is white noise with spectral density $S_w(\omega) = q$. Equivalently, this model can be written in Itô form as

$$d\mathbf{x}(t) = \mathbf{F}\mathbf{x}(t)\,dt + \mathbf{L}\,d\beta(t), \tag{4.2}$$

where $\beta(t)$ is a Brownian motion satisfying

$$\mathbb{E}[d\beta(t)d\beta(t)] = q\,dt.$$

For the companion-form representation, the state is chosen as

$$\mathbf{x}(t) = \left(f(t), \frac{df(t)}{dt}, \ldots, \frac{d^{m-1}f(t)}{dt^{m-1}} \right)^\top.$$

The matrices $\mathbf{F} \in \mathbb{R}^{m \times m}$ and $\mathbf{L} \in \mathbb{R}^{m \times 1}$ are given by

$$\mathbf{F} = \begin{pmatrix} 0 & 1 & 0 & \cdots & 0 \\ 0 & 0 & 1 & \cdots & 0 \\ \vdots & & \ddots & \ddots & \vdots \\ 0 & \cdots & 0 & 0 & 1 \\ -a_0 & -a_1 & \cdots & -a_{m-2} & -a_{m-1} \end{pmatrix}, \qquad \mathbf{L} = \begin{pmatrix} 0 \\ 0 \\ \vdots \\ 0 \\ 1 \end{pmatrix}. \tag{4.3}$$

This is known as the *companion form* of the m-th order scalar LTI SDE

$$\frac{d^m f}{dt^m} + a_{m-1}\frac{d^{m-1} f}{dt^{m-1}} + \cdots + a_1 \frac{df}{dt} + a_0 f = w(t). \tag{4.4}$$

Defining

$$\mathbf{H} = \begin{pmatrix} 1 & 0 & \ldots & 0 \end{pmatrix},$$

we have $f(t) = \mathbf{H}\mathbf{x}(t)$, so that the first component of $\mathbf{x}(t)$ recovers the process of interest. The matrices $\mathbf{F}, \mathbf{L}$ and the scalar q are the quantities we wish to determine for the construction of the LTI SDE. The matrix $\mathbf{L}$ always has the same companion-form structure; only its dimension changes with m. In what follows, we derive the spectral density and covariance function of $f(t)$ in terms of these matrices and the scalar q.

4.2 Spectral Density and Covariance Function of the Output Process

We first derive an explicit formula for the spectral density of the output process $f(t)$.

Lemma 4.1 (Spectral Density of LTI SDE Output)**.** *Let* $\mathbf{x}(t)$ *satisfy* (4.2) *in the stationary state. Then the spectral density of* $f(t) = \mathbf{H}\mathbf{x}(t)$ *is given by*

$$S_f(\omega) = \mathbf{H}(\mathbf{F} - i\omega\mathbf{I})^{-1}\mathbf{L}\, q\, \mathbf{L}^\top (\mathbf{F} + i\omega\mathbf{I})^{-\top}\mathbf{H}^\top.$$

Proof. Since $f(t) = \mathbf{H}\mathbf{x}(t)$, we have $\hat{f}(\omega) = \mathbf{H}\hat{\mathbf{x}}(\omega)$, where $\hat{f}(\omega)$ and $\hat{\mathbf{x}}(\omega)$ denote the Fourier transforms of $f(t)$ and $\mathbf{x}(t)$, respectively. By definition, the spectral density of $f(t)$ satisfies

$$S_f(\omega) = \mathbb{E}\Big[\hat{f}(\omega)\hat{f}(-\omega)\Big] = \mathbb{E}\Big[\mathbf{H}\hat{\mathbf{x}}(\omega)\left(\mathbf{H}\hat{\mathbf{x}}(-\omega)\right)^\top\Big].$$

Since $\mathbf{H}$ does not depend on ω, we have

$$S_f(\omega) = \mathbf{H}\,\mathbb{E}\big[\hat{\mathbf{x}}(\omega)\hat{\mathbf{x}}(-\omega)^\top\big]\,\mathbf{H}^\top = \mathbf{H}\,S_{\mathbf{x}}(\omega)\,\mathbf{H}^\top, \tag{4.5}$$

where

$$S_{\mathbf{x}}(\omega) := \mathbb{E}\big[\hat{\mathbf{x}}(\omega)\hat{\mathbf{x}}(-\omega)^\top\big].$$

Taking the Fourier transform of (4.2) and using $\mathcal{F}\{d\mathbf{x}/dt\} = i\omega\hat{\mathbf{x}}(\omega)$, we obtain

$$\begin{aligned} i\omega\,\hat{\mathbf{x}}(\omega) &= \mathbf{F}\hat{\mathbf{x}}(\omega) + \mathbf{L}\hat{w}(\omega), \\ (i\omega\mathbf{I} - \mathbf{F})\hat{\mathbf{x}}(\omega) &= \mathbf{L}\hat{w}(\omega), \\ \hat{\mathbf{x}}(\omega) &= (i\omega\mathbf{I} - \mathbf{F})^{-1}\mathbf{L}\hat{w}(\omega). \end{aligned}$$

By the definition of white noise,

$$S_w(\omega) = \mathbb{E}[\hat{w}(\omega)\hat{w}(-\omega)] = q.$$

Thus,

$$\begin{aligned} S_{\mathbf{x}}(\omega) &= \mathbb{E}\big[(i\omega\mathbf{I} - \mathbf{F})^{-1}\mathbf{L}\hat{w}(\omega)\hat{w}(-\omega)\mathbf{L}^\top(-i\omega\mathbf{I} - \mathbf{F})^{-\top}\big] \\ &= (i\omega\mathbf{I} - \mathbf{F})^{-1}\mathbf{L}\,q\,\mathbf{L}^\top(-i\omega\mathbf{I} - \mathbf{F})^{-\top} \\ &= (\mathbf{F} - i\omega\mathbf{I})^{-1}\mathbf{L}\,q\,\mathbf{L}^\top(\mathbf{F} + i\omega\mathbf{I})^{-\top}. \end{aligned}$$

Substituting back into (4.5) gives

$$S_f(\omega) = \mathbf{H}(\mathbf{F} - i\omega\mathbf{I})^{-1}\mathbf{L}\,q\,\mathbf{L}^\top(\mathbf{F} + i\omega\mathbf{I})^{-\top}\mathbf{H}^\top. \tag{4.6}$$

□

We next express the covariance function $k_f(\tau)$ directly in terms of the stationary state covariance $\mathbf{P}_\infty$ of $\mathbf{x}(t)$.

Proposition 4.2 (Covariance Function of LTI SDE Output)**.** *Let $\mathbf{x}(t)$ satisfy (4.2) in the stationary state, and let $\Phi(\tau) = \exp(\mathbf{F}\tau)$. Then the covariance function of $f(t) = \mathbf{H}\mathbf{x}(t)$ is*

$$k_f(\tau) = \begin{cases} \mathbf{H}\,\mathbf{P}_\infty\,\Phi(\tau)^\top\,\mathbf{H}^\top, & \tau \geq 0, \\ \mathbf{H}\,\Phi(-\tau)\,\mathbf{P}_\infty\,\mathbf{H}^\top, & \tau < 0, \end{cases}$$

where $\mathbf{P}_\infty = \mathbb{E}[\mathbf{x}(t)\mathbf{x}(t)^\top]$ is the stationary covariance of $\mathbf{x}(t)$, satisfying the Lyapunov equation

$$\mathbf{F}\mathbf{P}_\infty + \mathbf{P}_\infty\mathbf{F}^\top + \mathbf{L}\,q\,\mathbf{L}^\top = 0. \tag{4.7}$$

Proof. Since $f(t) = \mathbf{H}\mathbf{x}(t)$, we have

$$\begin{aligned} k_f(\tau) &= \mathbb{E}\big[f(t)f(t+\tau)^\top\big] \\ &= \mathbf{H}\,\mathbb{E}\big[\mathbf{x}(t)\mathbf{x}(t+\tau)^\top\big]\,\mathbf{H}^\top \\ &= \mathbf{H}\,k_{\mathbf{x}}(\tau)\,\mathbf{H}^\top, \end{aligned} \tag{4.8}$$

so it suffices to find $k_{\mathbf{x}}(\tau) = \mathbb{E}[\mathbf{x}(t)\mathbf{x}(t+\tau)^\top]$. Solving (4.2) forward from time t gives, for $\tau \geq 0$,

$$\mathbf{x}(t+\tau) = \Phi(\tau)\,\mathbf{x}(t) + \eta(t,\tau), \tag{4.9}$$

where

$$\eta(t,\tau) = \int_t^{t+\tau} \exp(\mathbf{F}(t+\tau-s))\,\mathbf{L}\,d\beta(s).$$

Substituting (4.9) into the definition of $k_{\mathbf{x}}(\tau)$,

$$\begin{aligned} k_{\mathbf{x}}(\tau) &= \mathbb{E}\big[\mathbf{x}(t)\mathbf{x}(t+\tau)^\top\big] \\ &= \mathbb{E}\big[\mathbf{x}(t)\mathbf{x}(t)^\top\big]\,\Phi(\tau)^\top + \mathbb{E}\big[\mathbf{x}(t)\,\eta(t,\tau)^\top\big]. \end{aligned} \tag{4.10}$$

Since $\eta(t,\tau)$ depends only on Brownian increments after time t and $\mathbf{x}(t)$ depends on Brownian increments up to time t, they are independent, so the second term in (4.10) vanishes. Hence, for $\tau \geq 0$,

$$k_{\mathbf{x}}(\tau) = \mathbb{E}\big[\mathbf{x}(t)\mathbf{x}(t)^\top\big]\,\Phi(\tau)^\top = \mathbf{P}_\infty\,\Phi(\tau)^\top. \tag{4.11}$$

For $\tau < 0$, by stationarity,

$$k_{\mathbf{x}}(\tau) = k_{\mathbf{x}}(-\tau)^\top = \Phi(-\tau)\,\mathbf{P}_\infty. \tag{4.12}$$

Substituting (4.11) and (4.12) back into (4.8) yields the stated formula. To justify the Lyapunov equation, define

$$\mathbf{P}(t) := \mathbb{E}[\mathbf{x}(t)\mathbf{x}(t)^\top].$$

Using Itô's product rule (Lemma 2.8) and substituting the SDE gives

$$\begin{aligned} d(\mathbf{x}(t)\mathbf{x}(t)^\top) &= \mathbf{F}\mathbf{x}(t)\mathbf{x}(t)^\top\,dt + \mathbf{L}\,d\beta(t)\,\mathbf{x}(t)^\top \\ &\quad + \mathbf{x}(t)\mathbf{x}(t)^\top\mathbf{F}^\top\,dt + \mathbf{x}(t)\,d\beta(t)\,\mathbf{L}^\top \\ &\quad + \mathbf{L}\,d\beta(t)\,d\beta(t)\,\mathbf{L}^\top. \end{aligned}$$

Taking expectations, the terms linear in $d\beta(t)$ vanish because

$$\mathbb{E}[d\beta(t)] = 0,$$

while, since β is a scalar Brownian motion of intensity q, the Itô rule gives

$$\mathbb{E}[d\beta(t)\,d\beta(t)] = q\,dt.$$

Hence,

$$d\mathbf{P}(t) = \big(\mathbf{F}\mathbf{P}(t) + \mathbf{P}(t)\mathbf{F}^\top + \mathbf{L}q\mathbf{L}^\top\big)\,dt,$$

so that

$$\frac{d\mathbf{P}(t)}{dt} = \mathbf{F}\mathbf{P}(t) + \mathbf{P}(t)\mathbf{F}^\top + \mathbf{L}q\mathbf{L}^\top.$$

In the stationary case, $\mathbf{P}(t) = \mathbf{P}_\infty$ is constant in time, and therefore

$$\mathbf{F}\mathbf{P}_\infty + \mathbf{P}_\infty\mathbf{F}^\top + \mathbf{L}q\mathbf{L}^\top = 0.$$

□

4.3 Rationality of the Spectral Density

We now provide the key structure property of $S_f(\omega)$.

Lemma 4.3 (Rationality of the Spectral Density)**.** *Let $S_f(\omega)$ be the spectral density of $f(t) = \mathbf{H}\mathbf{x}(t)$ as in Lemma 4.1. Then*

$$S_f(\omega) = \frac{q}{\prod_{k=1}^{m}(\lambda_k^2 + \omega^2)},$$

where $\lambda_1, \ldots, \lambda_m$ are the eigenvalues of $\mathbf{F}$. In particular, $S_f(\omega)$ is a rational function of ω^2.

Proof. Let $\mathbf{F}$ be the companion matrix in (4.3), and define

$$\mathbf{M} = \mathbf{F} - i\omega\mathbf{I} = \begin{pmatrix} -i\omega & 1 & 0 & \cdots & 0 \\ 0 & -i\omega & 1 & \vdots & \vdots \\ \vdots & \ddots & \ddots & \ddots & \vdots \\ 0 & \cdots & 0 & -i\omega & 1 \\ -a_0 & -a_1 & \cdots & -a_{m-2} & -a_{m-1} - i\omega \end{pmatrix},$$

By Lemma 4.1,

$$S_f(\omega) = \mathbf{H}\mathbf{M}^{-1}\mathbf{L} \cdot q \cdot \mathbf{L}^\top \bar{\mathbf{M}}^{-\top}\mathbf{H}^\top,$$

where $\bar{\mathbf{M}} = \mathbf{F} + i\omega\mathbf{I}$. Writing $\mathbf{M}^{-1} = \frac{1}{\det(\mathbf{M})}\operatorname{adj}(\mathbf{M})$, we have that

$$\mathbf{H}\mathbf{M}^{-1}\mathbf{L} = \frac{1}{\det(\mathbf{M})}\left[\operatorname{adj}(\mathbf{M})\right]_{1,m},$$

because multiplying by $\mathbf{H}$ on the left and $\mathbf{L}$ on the right selects the first row and last column. We can find this using the following formula:

$$\left[\operatorname{adj}(\mathbf{M})\right]_{1,m} = (-1)^{1+m}\det(\mathbf{M}_{m,1})$$

where $M_{m,1}$ denotes the $(m,1)$-minor of M, that is, the submatrix obtained by deleting the m-th row and first column of M. Then

$$\begin{aligned} \left[\operatorname{adj}(\mathbf{M})\right]_{1,m} &= (-1)^{1+m}\det\begin{pmatrix} 1 & 0 & \cdots & \cdots & 0 \\ -i\omega & 1 & 0 & \cdots & \vdots \\ 0 & -i\omega & 1 & \ddots & \vdots \\ \vdots & \ddots & \ddots & \ddots & \vdots \\ 0 & & \cdots & -i\omega & 1 \end{pmatrix} \\ &= (-1)^{1+m} \cdot 1 \\ &= (-1)^{m+1}. \end{aligned}$$

For the determinant we use the factorization of the characteristic polynomial:

$$\det(\mathbf{M}) = \det(\mathbf{F} - i\omega\mathbf{I}) = \prod_{k=1}^{m}(\lambda_k - i\omega),$$

where $\lambda_1, \ldots, \lambda_m$ are the eigenvalues of $\mathbf{F}$. Applying the same argument to $\bar{\mathbf{M}}$ gives

$$\left[\mathrm{adj}(\bar{\mathbf{M}})\right]_{1,m} = (-1)^{m+1} \quad \text{and} \quad \det(\bar{\mathbf{M}}) = \prod_{k=1}^{m}(\lambda_k + i\omega).$$

Combining these results, and noting that $(-1)^{m+1}\cdot(-1)^{m+1} = (-1)^{2(m+1)} = 1$, we obtain

$$\begin{aligned} S_f(\omega) &= \mathbf{H}\mathbf{M}^{-1}\mathbf{L} \cdot q \cdot \left(\mathbf{H}\bar{\mathbf{M}}^{-1}\mathbf{L}\right)^{\top} \\ &= \frac{(-1)^{m+1}}{\prod_{k=1}^{m}(\lambda_k - i\omega)} \cdot q \cdot \frac{(-1)^{m+1}}{\prod_{k=1}^{m}(\lambda_k + i\omega)}. \end{aligned}$$

Since $\mathbf{F}$ is real, its non-real eigenvalues occur in complex conjugate pairs [11]. A real eigenvalue λ_k contributes the factor $\lambda_k^2 + \omega^2$ directly, which is already a real polynomial in ω^2. If instead λ and $\bar{\lambda}$ form a conjugate pair, then $(\lambda^2 + \omega^2)(\bar{\lambda}^2 + \omega^2) = |\lambda|^4 + 2\mathrm{Re}(\lambda^2)\omega^2 + \omega^4$, which is again a real polynomial in ω^2. Therefore, although an individual factor $\lambda_k^2 + \omega^2$ may be complex when λ_k is complex, the product over each conjugate pair is real and contains only even powers of ω, while the real eigenvalues contribute real factors. Hence

$$\prod_{k=1}^{m}(\lambda_k^2 + \omega^2)$$

is a real polynomial in ω^2. Therefore $S_f(\omega)$ is rational in ω^2. □

Lemma 4.3 is the central result of this section. It shows that any LTI SDE of the companion form can only produce processes whose spectral density is rational in ω^2. This motivates the following strategy: given a target kernel $k(\tau)$ whose spectral density $S(\omega)$ is also rational in ω^2, we can run the argument in reverse, using spectral factorization to recover $\mathbf{F}$, $\mathbf{L}$, and q directly from $S(\omega)$. The next section makes this precise.

4.4 Constructing an LTI SDE from a Covariance Function

We now run the argument in reverse. The following proposition shows that whenever we are given a stationary covariance whose spectral density is rational in ω^2, we can construct an LTI SDE that exactly matches the covariance.

Proposition 4.4 (SDE Construction via Spectral Factorization)**.** *Let $k(\tau)$ be a stationary covariance function whose spectral density is of the form*

$$S(\omega) = \frac{c}{\prod_{k=1}^{m}(\lambda_k^2 + \omega^2)}$$

where $c > 0$, $\mathrm{Re}(\lambda_k) > 0$ for all $k \in \{1, \cdots, m\}$, and complex λ_k occur in conjugate pairs. Define the polynomial

$$P(i\omega) = \prod_{k=1}^{m}(\lambda_k + i\omega) := (i\omega)^m + h_{m-1}(i\omega)^{m-1} + \cdots + h_0$$

with $h_0, \cdots, h_{m-1} \in \mathbb{R}$. Then the LTI SDE with matrices

$$\mathbf{F} = \begin{pmatrix} 0 & 1 & 0 & \cdots & 0 \\ 0 & 0 & 1 & \cdots & 0 \\ \vdots & & \ddots & \ddots & \vdots \\ 0 & \cdots & 0 & 0 & 1 \\ -h_0 & -h_1 & \cdots & -h_{m-2} & -h_{m-1} \end{pmatrix}, \quad \mathbf{L} = \begin{pmatrix} 0 \\ \vdots \\ 0 \\ 1 \end{pmatrix}, \quad q = c,$$

has output $f(t) = \mathbf{H}\mathbf{x}(t)$ with spectral density $S(\omega)$ and covariance function $k(\tau)$.

Proof. Since $\lambda_k^2 + \omega^2 = (\lambda_k + i\omega)(\lambda_k - i\omega)$, it follows that

$$S(\omega) = \frac{c}{\prod_{k=1}^{m}(\lambda_k + i\omega)(\lambda_k - i\omega)} = G(i\omega)\, c\, G(-i\omega) \tag{4.13}$$

where $G(i\omega) = 1/P(i\omega)$. Viewing P as a polynomial in the complex variable $z = i\omega$, we have $P(z) = \prod_{k=1}^{m}(z + \lambda_k)$, whose roots $z = -\lambda_k$ all have strictly negative real parts since $\mathrm{Re}(\lambda_k) > 0$. This places all poles of $G(z) = 1/P(z)$ in the left half of the complex z-plane, so G corresponds to a stable (causal) Markov process [7, 16]. From this comes the frequency-domain representation $\hat{f}(\omega) = G(i\omega)\hat{w}(\omega)$ which is equivalent to $P(i\omega)\hat{f}(\omega) = \hat{w}(\omega)$. After expanding, we get

$$(i\omega)^m \hat{f}(\omega) + h_{m-1}(i\omega)^{m-1}\hat{f}(\omega) + \cdots + h_0\hat{f}(\omega) = \hat{w}(\omega).$$

Since the λ_k occur in conjugate pairs when complex, the coefficients $h_0, \ldots, h_{m-1}$ are real, by the same elementary-symmetric-polynomial argument as in Lemma 4.3. Applying the differentiation property $\mathcal{F}(d^n f/dt^n) = (i\omega)^n \hat{f}(\omega)$, this becomes, in the time domain,

$$\frac{d^m f}{dt^m} + h_{m-1}\frac{d^{m-1} f}{dt^{m-1}} + \cdots + h_1\frac{df}{dt} + h_0 f = w(t).$$

This is exactly the companion form (4.4) where $a_j = h_j$ for all $j \in \{0, \cdots, m-1\}$. Thus the coefficients of $P(i\omega)$ are precisely the entries of the companion matrix $\mathbf{F}$. □

The result of this section is therefore a continuous-time state-space representation of the Gaussian process prior. Starting from a stationary covariance function with rational spectral density, spectral factorization gives the matrices $\mathbf{F}$, $\mathbf{L}$, the observation matrix $\mathbf{H}$, and the noise intensity q. The output process

$$f(t) = \mathbf{H}\mathbf{x}(t)$$

then has the desired covariance function $k(\tau)$. In the next chapter, we discretize this continuous-time model at the observation times and use Bayesian filtering and smoothing to compute the corresponding posterior distributions.

5 State-Space Inference

So far, we have constructed a continuous linear time-invariant stochastic differential equation whose output has the same covariance function as the target stationary Gaussian process prior. In what follows, we discretize this continuous-time model, which gives a linear Gaussian state-space model in which inference can be performed by Kalman filtering and Rauch–Tung–Striebel smoothing.

The role of filtering and smoothing should be distinguished carefully. Kalman filtering computes the posterior distribution of the state at time t_k using only the observations up to that time, namely

$$p(\mathbf{x}_k \mid y_{1:k}).$$

In contrast, Gaussian process regression conditions on the full dataset. To obtain the corresponding state-space posterior at time t_k, one applies RTS smoothing after the filtering pass, which gives

$$p(\mathbf{x}_k \mid y_{1:T}).$$

The posterior distribution of the function value is then obtained from the state through

$$f(t_k) = \mathbf{H}\mathbf{x}_k.$$

Thus, the state-space formulation replaces the dense Gaussian process regression computation with forward filtering and backward smoothing recursions.

5.1 Discretization of the Continuous-Time State-Space Model

The first step is to convert the continuous-time model into a discrete-time state-space model. Starting from (4.2), the exact solution from t_{k-1} to t_k is given by the standard solution formula for linear SDEs [16]

$$\mathbf{x}(t_k) = \exp(\mathbf{F}\Delta t_k)\mathbf{x}(t_{k-1}) + \int_{t_{k-1}}^{t_k} \exp(\mathbf{F}(t_k - s))\,\mathbf{L}\,d\beta(s),$$

where

$$\Delta t_k := t_k - t_{k-1}.$$

Define

$$\mathbf{x}_k := \mathbf{x}(t_k), \qquad \mathbf{A}_{k-1} := \exp(\mathbf{F}\Delta t_k).$$

Then the discrete-time state equation can be written as

$$\mathbf{x}_k = \mathbf{A}_{k-1}\mathbf{x}_{k-1} + \mathbf{q}_{k-1},$$

where

$$\mathbf{q}_{k-1} := \int_{t_{k-1}}^{t_k} \exp(\mathbf{F}(t_k - s))\,\mathbf{L}\,d\beta(s).$$

We now rewrite the stochastic integral over an interval starting at zero. Let

$$\tau = s - t_{k-1}.$$

Then

$$s = t_{k-1} + \tau, \qquad d\tau = ds, \qquad \tau \in [0, \Delta t_k],$$

and

$$t_k - s = t_k - (t_{k-1} + \tau) = \Delta t_k - \tau.$$

Therefore,

$$\mathbf{q}_{k-1} = \int_0^{\Delta t_k} \exp(\mathbf{F}(\Delta t_k - \tau))\,\mathbf{L}\,d\beta(t_{k-1} + \tau).$$

Define the shifted Brownian increment process

$$\widetilde{\beta}_k(\tau) := \beta(t_{k-1} + \tau) - \beta(t_{k-1}), \qquad 0 \le \tau \le \Delta t_k.$$

Then

$$d\widetilde{\beta}_k(\tau) = d\beta(t_{k-1} + \tau),$$

and hence

$$\mathbf{q}_{k-1} = \int_0^{\Delta t_k} \exp(\mathbf{F}(\Delta t_k - \tau))\,\mathbf{L}\,d\widetilde{\beta}_k(\tau).$$

Since the integrand is deterministic and the Brownian increment has mean zero, the Itô integral has mean zero:

$$\mathbb{E}[\mathbf{q}_{k-1}] = \mathbf{0}.$$

The covariance is

$$\mathbf{Q}_{k-1} := \mathbb{E}\left[\mathbf{q}_{k-1}\mathbf{q}_{k-1}^\top\right].$$

Using the Itô isometry (Lemma 2.9),

$$\mathbf{Q}_{k-1} = \int_0^{\Delta t_k} \exp(\mathbf{F}(\Delta t_k - \tau))\,\mathbf{L}\,q\,\mathbf{L}^\top \exp(\mathbf{F}(\Delta t_k - \tau))^\top\,d\tau.$$

Here q appears because the driving Brownian motion has intensity q, so

$$\mathbb{E}\left[\left(d\widetilde{\beta}_k(\tau)\right)^2\right] = q\,d\tau.$$

Remark. Once $\mathbf{P}_\infty$ has been computed by solving (4.7), forming the equivalent discretization becomes straightforward. The discrete-time version of the Lyapunov equation (4.7) is

$$\mathbf{P}_\infty = \mathbf{A}_{k-1}\mathbf{P}_\infty\mathbf{A}_{k-1}^\top + \mathbf{Q}_{k-1}, \tag{5.1}$$

which upon rearranging gives

$$\mathbf{Q}_{k-1} = \mathbf{P}_\infty - \mathbf{A}_{k-1}\mathbf{P}_\infty\mathbf{A}_{k-1}^\top. \tag{5.2}$$

This provides a computationally convenient alternative to evaluating the Itô isometry integral directly, and will be used in the Matérn example of Section 6.2 [16].

Therefore,

$$\mathbf{q}_{k-1} \sim \mathcal{N}(\mathbf{0}, \mathbf{Q}_{k-1}),$$

and the exact discrete-time model is

$$\mathbf{x}_k = \mathbf{A}_{k-1}\mathbf{x}_{k-1} + \mathbf{q}_{k-1}, \qquad \mathbf{q}_{k-1} \sim \mathcal{N}(\mathbf{0}, \mathbf{Q}_{k-1}). \tag{5.3}$$

To match the stationary Gaussian process prior, the initial state is chosen from the stationary distribution of the continuous-time SDE:

$$\mathbf{x}_0 \sim \mathcal{N}(\mathbf{0}, \mathbf{P}_\infty),$$

where $\mathbf{P}_\infty$ solves the continuous-time Lyapunov equation (4.7). With this choice, the covariance of the state-space model agrees with the covariance function of the Gaussian process prior from the initial time onward. The measurement model is of the form

$$y_k = \mathbf{H}\mathbf{x}_k + \varepsilon_k, \qquad \varepsilon_k \sim \mathcal{N}(0, \sigma^2_{\text{noise}}). \tag{5.4}$$

This is the standard linear Gaussian measurement model used in the state-space formulation of temporal Gaussian process regression [7].

5.2 Bayesian Filtering Equations

After discretizing the continuous-time LTI SDE, we obtain a probabilistic state-space model of the form

$$p(\mathbf{x}_k \mid \mathbf{x}_{k-1}) \qquad \text{and} \qquad p(y_k \mid \mathbf{x}_k).$$

The state process is Markovian, meaning that the next state depends on the past only through the current state. Moreover, the measurement y_k, given $\mathbf{x}_k$, is conditionally independent of the previous states and measurements. These are the assumptions under which the Bayesian filtering equations apply [17].

Theorem 5.1 (Bayesian filtering equations)**.** *The recursive equations for computing the predicted distribution $p(\mathbf{x}_k \mid y_{1:k-1})$ and the filtering distribution $p(\mathbf{x}_k \mid y_{1:k})$ at time step k are given as follows [17]:*

1. *Initialization: Choose a prior distribution* $p(\mathbf{x}_0)$.

2. *Prediction step: The predictive distribution of the state* $\mathbf{x}_k$ *at time step* k*, given the dynamical model, can be computed using the Chapman–Kolmogorov equation*

$$p(\mathbf{x}_k \mid y_{1:k-1}) = \int p(\mathbf{x}_k \mid \mathbf{x}_{k-1}) p(\mathbf{x}_{k-1} \mid y_{1:k-1})\, d\mathbf{x}_{k-1}.$$

3. *Update step: Given the measurement* y_k *at time step* k*, the posterior distribution of the state* $\mathbf{x}_k$ *can be computed by Bayes' rule*

$$p(\mathbf{x}_k \mid y_{1:k}) = \frac{p(y_k \mid \mathbf{x}_k)\, p(\mathbf{x}_k \mid y_{1:k-1})}{\int p(y_k \mid \mathbf{x}_k)\, p(\mathbf{x}_k \mid y_{1:k-1})\, d\mathbf{x}_k}.$$

5.3 The Kalman Filter

The Kalman filter, invented in the 1960s by Kalman [8], is a closed-form solution to the Bayesian filtering equations for the specific LTI model (5.3). In our case

$$\mathbf{x}_k \mid \mathbf{x}_{k-1} \sim \mathcal{N}(\mathbf{x}_k \mid \mathbf{A}_{k-1}\mathbf{x}_{k-1}, \mathbf{Q}_{k-1})$$

and

$$y_k \mid \mathbf{x}_k \sim \mathcal{N}(y_k \mid \mathbf{H}\mathbf{x}_k, \sigma^2_{\text{noise}}).$$

The Kalman filter finds the following:

$$\begin{aligned} \mathbf{x}_k \mid y_{1:k-1} &\sim \mathcal{N}(\mathbf{x}_k \mid \mathbf{m}_k^-, \mathbf{P}_k^-), \\ \mathbf{x}_k \mid y_{1:k} &\sim \mathcal{N}(\mathbf{x}_k \mid \mathbf{m}_k, \mathbf{P}_k), \\ y_k \mid y_{1:k-1} &\sim \mathcal{N}(y_k \mid \mathbf{H}\mathbf{m}_k^-, S_k). \end{aligned}$$

The parameters of the distributions above are computed with the following prediction and update steps.

Prediction step:

$$\begin{aligned} \mathbf{m}_k^- &= \mathbf{A}_{k-1}\mathbf{m}_{k-1}, \\ \mathbf{P}_k^- &= \mathbf{A}_{k-1}\mathbf{P}_{k-1}\mathbf{A}_{k-1}^\top + \mathbf{Q}_{k-1}. \end{aligned} \tag{5.5}$$

Update step:

$$\begin{aligned} v_k &= y_k - \mathbf{H}\mathbf{m}_k^-, \\ S_k &= \mathbf{H}\mathbf{P}_k^-\mathbf{H}^\top + \sigma^2_{\text{noise}}, \\ \mathbf{K}_k &= \mathbf{P}_k^-\mathbf{H}^\top S_k^{-1}, \\ \mathbf{m}_k &= \mathbf{m}_k^- + \mathbf{K}_k v_k, \\ \mathbf{P}_k &= \mathbf{P}_k^- - \mathbf{K}_k S_k \mathbf{K}_k^\top. \end{aligned} \tag{5.6}$$

These are the standard Kalman filter prediction and update recursions for a linear Gaussian state-space model [17]. The filter processes observations one at a time, which is illustrated in Figure 5.1.

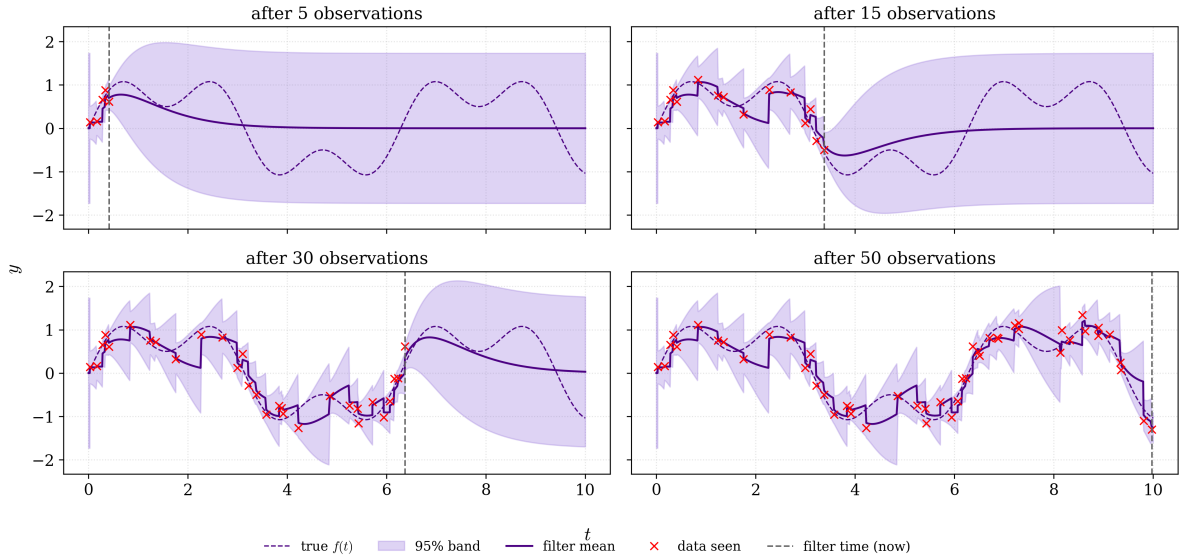


Figure 5.1: Kalman filter posterior after sequentially processing 5, 15, 30, and 50 observations. The posterior at each stage is conditioned only on the observations already seen, so the process remains at prior uncertainty ahead of the latest one.

The Kalman filter therefore gives the filtering distribution

$$p(\mathbf{x}_k \mid y_{1:k}),$$

which uses only the observations up to time t_k. The corresponding posterior distribution of the function value is obtained from

$$f(t_k) = \mathbf{H}\mathbf{x}_k.$$

However, standard Gaussian process regression conditions on the full observation vector. To obtain the state-space posterior conditioned on all observations, one applies a smoothing pass after filtering. In the linear Gaussian case considered here, this is done by the Rauch–Tung–Striebel smoother.

5.4 Bayesian Smoothing Equations

We now consider the Bayesian smoothing equations. The goal of smoothing is to compute the marginal posterior distribution of the state $\mathbf{x}_k$ at time step k after receiving measurements up to time step T, where $T > k$:

$$p(\mathbf{x}_k \mid y_{1:T}).$$

This is the state-space analogue of conditioning on the full dataset in Gaussian process regression. In practice, smoothing is performed after a forward filtering pass, and the resulting recursion runs backward in time.

Theorem 5.2 (Bayesian Smoothing Equations)**.** *The Bayesian smoothing recursion is given by [17]*

$$p(\mathbf{x}_{k+1} \mid y_{1:k}) = \int p(\mathbf{x}_{k+1} \mid \mathbf{x}_k) p(\mathbf{x}_k \mid y_{1:k}) \, d\mathbf{x}_k$$

and

$$p(\mathbf{x}_k \mid y_{1:T}) = p(\mathbf{x}_k \mid y_{1:k}) \int \frac{p(\mathbf{x}_{k+1} \mid \mathbf{x}_k) p(\mathbf{x}_{k+1} \mid y_{1:T})}{p(\mathbf{x}_{k+1} \mid y_{1:k})} \, d\mathbf{x}_{k+1}.$$

5.5 Rauch–Tung–Striebel Smoother

The Rauch–Tung–Striebel smoother [15], also known as the Kalman smoother, is again a closed-form smoothing solution

$$\mathbf{x}_k \mid y_{1:T} \sim \mathcal{N}(\mathbf{x}_k \mid \mathbf{m}_k^s, \mathbf{P}_k^s)$$

to the LTI model (5.3).

The backward recursion equations for the RTS smoother are given as

$$\begin{aligned} \mathbf{m}_{k+1}^- &= \mathbf{A}_k \mathbf{m}_k, \\ \mathbf{P}_{k+1}^- &= \mathbf{A}_k \mathbf{P}_k \mathbf{A}_k^\top + \mathbf{Q}_k, \\ \mathbf{G}_k &= \mathbf{P}_k \mathbf{A}_k^\top (\mathbf{P}_{k+1}^-)^{-1}, \\ \mathbf{m}_k^s &= \mathbf{m}_k + \mathbf{G}_k (\mathbf{m}_{k+1}^s - \mathbf{m}_{k+1}^-), \\ \mathbf{P}_k^s &= \mathbf{P}_k + \mathbf{G}_k (\mathbf{P}_{k+1}^s - \mathbf{P}_{k+1}^-) \mathbf{G}_k^\top. \end{aligned} \tag{5.7}$$

Here $\mathbf{m}_k$ and $\mathbf{P}_k$ are the mean and covariance computed by the Kalman filter. It is initialized at the final time step T, with $\mathbf{m}_T^s = \mathbf{m}_T$ and $\mathbf{P}_T^s = \mathbf{P}_T$. The effect of the backward pass is illustrated in Figure 5.2.

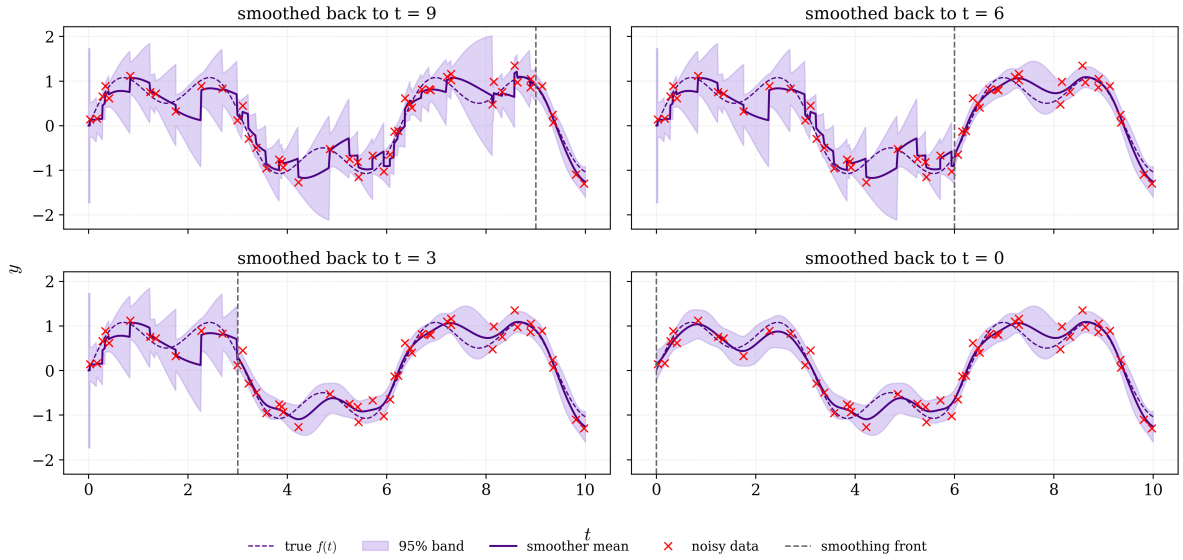


Figure 5.2: RTS smoother backward pass, shown at $t = 9,\ 6,\ 3,\ 0$. Starting from the filtered estimate at the final time, each backward step re-conditions earlier states on the entire dataset, so the posterior tightens relative to the forward filter of Figure 5.1.

5.6 The Computational Advantage

This completes the connection between the state-space representation and Bayesian inference. The computational bottleneck identified in Chapter 3 was the need to work directly with the dense covariance matrix

$$\mathbf{K} + \sigma^2_{\text{noise}}\mathbf{I}_n$$

of all observations. In the state-space formulation, this dense matrix is not formed or factorized directly. Instead, posterior inference is carried out by a forward Kalman filtering pass followed by a backward RTS smoothing pass.

If n denotes the number of observations and m the dimension of the state vector, the cost of Kalman filtering and RTS smoothing is $\mathcal{O}(nm^3)$. In the temporal Gaussian process models considered here, m is fixed by the order of the state-space representation and is typically small. Therefore, the computational cost is linear in n in practice. This should be compared with direct Gaussian process regression with a dense covariance matrix, which requires $\mathcal{O}(n^3)$ computation and $\mathcal{O}(n^2)$ memory [7, 16, 14].

5.7 Hyperparameter Learning

Throughout we have treated the hyperparameters θ as fixed and given. We now show that the same forward pass that performs filtering also returns, at no extra cost, the standard objective used to select them.

In Gaussian process regression it is customary to choose the hyperparameters by maximizing the marginal likelihood of the data,

$$p(y_{1:n} \mid \theta) = \prod_{k=1}^{n} p(y_k \mid y_{1:k-1}, \theta).$$

The principle is that one prefers the θ under which the observed data is least surprising, that is, assigned the highest density [14]. Computing this quantity directly is subject to the same bottleneck as inference: the dense marginal likelihood again requires the factorization of an $n \times n$ matrix and so costs $\mathcal{O}(n^3)$. The Kalman filter, by contrast, produces each factor for free.

Indeed, the one-step predictive density $p(y_k \mid y_{1:k-1}, \theta)$ is exactly the distribution of $y_k \mid y_{1:k-1}$ already returned by the filter,

$$y_k \mid y_{1:k-1}, \theta \sim \mathcal{N}\big(\mathbf{H}\mathbf{m}_k^-,\, S_k\big)\,,$$

with mean $\mathbf{H}\mathbf{m}_k^-$ and variance $S_k = \mathbf{H}\mathbf{P}_k^-\mathbf{H}^\top + \sigma^2_{\text{noise}}$. Both the innovation $v_k = y_k - \mathbf{H}\mathbf{m}_k^-$ and the innovation variance S_k are computed by the update step (5.6) regardless, since the Kalman gain is built from them. The marginal likelihood is therefore assembled from quantities the recursion already produces.

One maximizes the marginal likelihood by minimizing its negative logarithm. Substituting the predictive density into the factorization and taking negative logarithms gives the energy function

$$-\log p(y_{1:n} \mid \theta) = \frac{n}{2}\log(2\pi) + \frac{1}{2}\sum_{k=1}^{n}\left(\log S_k + \frac{v_k^2}{S_k}\right),$$

which is accumulated as a running sum during the forward pass [10]. Since filtering is $\mathcal{O}(nm^3)$, evaluating this objective is $\mathcal{O}(nm^3)$ as well, and the gradient with respect to θ, obtained by differentiating the recursion, is of the same order. The linear-time advantage thus extends from inference to hyperparameter learning. Whenever the kernel admits an exact state-space representation, as the Matérn family does, the resulting estimate coincides with the one obtained from the dense marginal likelihood; the hyperparameters used in our experiments (Section 7.1) are fitted in exactly this way, there through the dense computation for simplicity.

6 The Construction Recipe and the Matérn Covariance Family

The preceding chapters have developed all the ingredients needed to convert a Gaussian process regression problem into an equivalent state-space inference problem. In this chapter we bring these ingredients together. We first present the construction as a self-contained recipe, and then apply it to the Matérn covariance family, which is the best-known class of covariance functions for which the spectral density is rational in ω^2, and for which the construction therefore yields an exact finite-dimensional state-space representation.

6.1 The Construction Recipe

We now summarize the construction developed in the previous chapters. Suppose we are given a one-dimensional Gaussian process regression model with a stationary covariance function $k(\tau)$. The goal is to replace the Gaussian process prior by an equivalent linear time-invariant state-space model that can be handled by Kalman filtering and Rauch–Tung–Striebel smoothing.

The procedure is as follows.

1. Start from the stationary covariance function $k(\tau)$ of the Gaussian process prior and compute its spectral density $S(\omega)$. This can be done by taking the Fourier transform of $k(\tau)$.

2. Check whether the spectral density can be written in the rational form

$$S(\omega) = \frac{c}{\text{polynomial in } \omega^2}.$$

   This is the class of spectral densities that can be represented by an LTI SDE via spectral factorization.

3. Factorize the spectral density as

$$S(\omega) = G(i\omega)\, q\, G(-i\omega),$$

   where $G(i\omega) = 1/P(i\omega)$ and the polynomial $P(i\omega)$ is chosen so that its roots lie in the left half-plane. Writing

$$P(i\omega) = (i\omega)^m + h_{m-1}(i\omega)^{m-1} + \cdots + h_0,$$

the coefficients $h_0, \ldots, h_{m-1}$ determine the last row of the companion matrix

$$\mathbf{F} = \begin{pmatrix} 0 & 1 & 0 & \cdots & 0 \\ 0 & 0 & 1 & \cdots & 0 \\ \vdots & & \ddots & \ddots & \vdots \\ 0 & \cdots & 0 & 0 & 1 \\ -h_0 & -h_1 & \cdots & -h_{m-2} & -h_{m-1} \end{pmatrix}.$$

Together with

$$\mathbf{L} = \begin{pmatrix} 0 \\ \vdots \\ 0 \\ 1 \end{pmatrix}, \qquad \mathbf{H} = \begin{pmatrix} 1 & 0 & \cdots & 0 \end{pmatrix},$$

this gives an LTI SDE whose output $f(t) = \mathbf{H}\mathbf{x}(t)$ has the same spectral density, and hence the same covariance function, as the original Gaussian process prior.

4. Compute the stationary covariance matrix $\mathbf{P}_\infty$ of the state by solving the continuous-time Lyapunov equation

$$\mathbf{F}\mathbf{P}_\infty + \mathbf{P}_\infty \mathbf{F}^\top + \mathbf{L} q \mathbf{L}^\top = 0.$$

This matrix provides the stationary prior needed to initialize the state-space model.

5. Discretize the continuous-time model at the observation times to obtain the corresponding discrete-time state-space model. Then run the Kalman filter forward and the Rauch–Tung–Striebel smoother backward in order to compute the posterior quantities.

6. The resulting inference is linear in the number of observations n when the state dimension m is fixed. More precisely, Kalman filtering and RTS smoothing scale as $\mathcal{O}(nm^3)$, and since m is typically small, this is $\mathcal{O}(n)$ in practice.

Thus, whenever the covariance function of the Gaussian process prior has a spectral density that is rational in ω^2, the Gaussian process regression problem can be reformulated as inference in a linear Gaussian state-space model. This gives a model with the same prior covariance structure, but with computational cost that is linear in the number of observations in practice.

6.2 The Matérn Covariance Family

In this section we carry out the full construction for the Matérn covariance family, introduced by Bertil Matérn in 1960. It is a stationary covariance function of the form [14, 12]

$$k_\nu(\tau) = \sigma^2 \frac{2^{1-\nu}}{\Gamma(\nu)} \left(\frac{\sqrt{2\nu}}{\ell} \tau \right)^\nu K_\nu \left(\frac{\sqrt{2\nu}}{\ell} \tau \right), \tag{6.1}$$

where $\ell > 0$ and $\sigma^2 > 0$ are the length-scale and magnitude hyperparameters, K_ν is the modified Bessel function of the second kind, and $\nu > 0$ controls the smoothness of the process. The spectral density of (6.1) for one-dimensional processes is [14]

$$S(\omega) = \sigma^2 \frac{2\sqrt{\pi}\,\Gamma(\nu + 1/2)}{\Gamma(\nu)} \lambda^{2\nu} (\lambda^2 + \omega^2)^{-(\nu+1/2)},$$

where $\lambda = \sqrt{2\nu}/\ell$. In this thesis we restrict to the case $\nu = p + 1/2$ for $p \in \mathbb{Z}^{\geq 0}$, under which the spectral density simplifies to

$$S(\omega) = \sigma^2 \frac{2\sqrt{\pi}\,\Gamma(p+1)}{\Gamma(p+1/2)} \lambda^{2p+1} (\lambda^2 + \omega^2)^{-(p+1)}.$$

This is precisely of the rational form required by Proposition 4.4, with

$$q = \sigma^2 \frac{2\sqrt{\pi}\,\Gamma(p+1)}{\Gamma(p+1/2)} \lambda^{2p+1}, \qquad G(i\omega) = (\lambda + i\omega)^{-(p+1)},$$

and hence an LTI SDE can be constructed via spectral factorization. Here the polynomial $(\lambda + i\omega)^{p+1}$ has the single repeated root $-\lambda$, so all $m = p + 1$ eigenvalues of $\mathbf{F}$ coincide at $-\lambda$ and the product $\prod_{k=1}^{m} (\lambda_k^2 + \omega^2)$ of Lemma 4.3 collapses to $(\lambda^2 + \omega^2)^{p+1}$. The state-space matrices for the first three values of p are as follows. For $p = 0$ the state is the scalar $x(t)$, and

$$\frac{dx(t)}{dt} = -\lambda\, x(t) + w(t), \qquad q = 2\lambda\sigma^2.$$

For $p = 1$,

$$\frac{d\mathbf{x}(t)}{dt} = \begin{pmatrix} 0 & 1 \\ -\lambda^2 & -2\lambda \end{pmatrix} \mathbf{x}(t) + \begin{pmatrix} 0 \\ 1 \end{pmatrix} w(t), \qquad q = 4\lambda^3\sigma^2.$$

For $p = 2$,

$$\frac{d\mathbf{x}(t)}{dt} = \begin{pmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ -\lambda^3 & -3\lambda^2 & -3\lambda \end{pmatrix} \mathbf{x}(t) + \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix} w(t), \qquad q = \frac{16}{3}\lambda^5\sigma^2.$$

In all cases $\mathbf{H} = \begin{pmatrix} 1 & 0 & \cdots & 0 \end{pmatrix}$. We now derive the discrete-time model for $p = 1$; the same procedure applies for any $p \in \mathbb{Z}^{\geq 0}$. We first solve the Lyapunov equation (4.7) for the stationary covariance $\mathbf{P}_\infty$:

$$\begin{pmatrix} 0 & 1 \\ -\lambda^2 & -2\lambda \end{pmatrix} \mathbf{P}_\infty + \mathbf{P}_\infty \begin{pmatrix} 0 & -\lambda^2 \\ 1 & -2\lambda \end{pmatrix} + \begin{pmatrix} 0 \\ 1 \end{pmatrix} 4\lambda^3\sigma^2 \begin{pmatrix} 0 & 1 \end{pmatrix} = 0,$$

which gives

$$\mathbf{P}_\infty = \begin{pmatrix} \sigma^2 & 0 \\ 0 & \lambda^2\sigma^2 \end{pmatrix}.$$

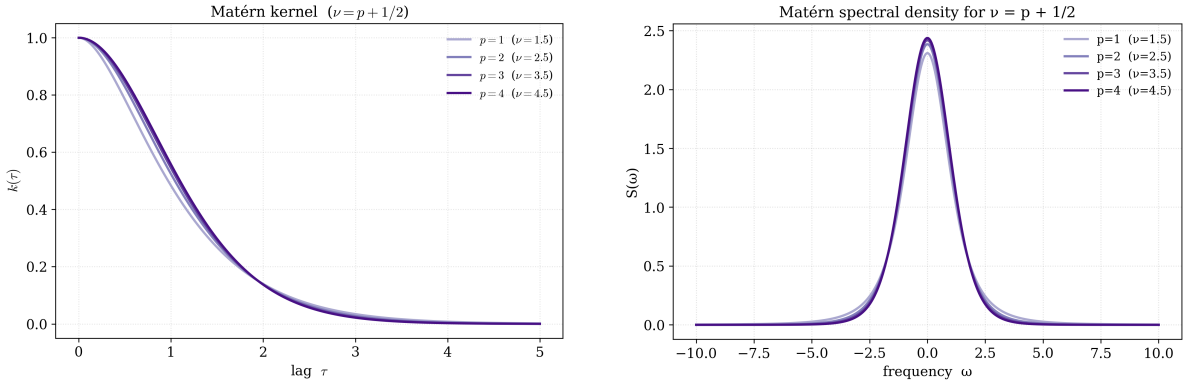
To discretize, we compute the transition matrix $\mathbf{A}_{k-1} = \exp(\mathbf{F}\Delta t_k)$ using the Jordan normal form of $\mathbf{F}$. Since $\mathbf{F}$ has a repeated eigenvalue $-\lambda$, by the Jordan block theorem [11]

$$\mathbf{A}_{k-1} = \exp(\mathbf{F}\Delta t_k) = e^{-\lambda\Delta t_k}\begin{pmatrix} 1+\lambda\Delta t_k & \Delta t_k \\ -\lambda^2\Delta t_k & 1-\lambda\Delta t_k \end{pmatrix}.$$

Finally, by the stationarity identity (5.2),

$$\mathbf{Q}_{k-1} = \sigma^2\begin{pmatrix} 1-e^{-2\lambda\Delta t_k}\left[(1+\lambda\Delta t_k)^2+\lambda^2\Delta t_k^2\right] & 2\lambda^3\Delta t_k^2\, e^{-2\lambda\Delta t_k} \\ 2\lambda^3\Delta t_k^2\, e^{-2\lambda\Delta t_k} & \lambda^2\left(1-e^{-2\lambda\Delta t_k}\left[(1-\lambda\Delta t_k)^2+\lambda^2\Delta t_k^2\right]\right) \end{pmatrix}.$$

Figure 6.1 shows the resulting Matérn covariance functions and their spectral densities for several smoothness orders.



(a) Covariance function $k_\nu(\tau)$.

(b) Spectral density $S(\omega)$.

Figure 6.1: Matérn covariance function and spectral density, shown for several smoothness orders $\nu = p + \frac{1}{2}$, $p \in \{1, 2, 3, 4\}$, with $\ell = 1.06$ and $\sigma^2 = 0.78$.

7 Experiments

In what follows we illustrate the efficiency of the proposed approach through a set of numerical experiments.[1]

7.1 Experimental Setup

All timings were measured on a single machine (Intel Core i7-1255U, 16 GB RAM, Windows 11), using Python 3.12 with scikit-learn 1.8, NumPy 2.4, and SciPy 1.17.

Both methods are run on identical data. We take the true function to be

$$f(t_i) = \sin(t_i) + 0.5\sin(3t_i),$$

and model the observations as in Chapter 3,

$$y_i = f(t_i) + \varepsilon_i, \qquad \varepsilon_i \sim \mathcal{N}(0, \sigma^2_{\text{noise}}),$$

with $\sigma_{\text{noise}} = 0.2$. The inputs t_i are drawn independently from a uniform distribution on $[0, 10]$ (random seed 0, for reproducibility) and sorted in increasing order. The number of observations depends on the experiment: the equivalence experiment below uses $n = 50$, whereas the scaling experiment that follows sweeps n in steps of 1000. Because the dense solve becomes prohibitively expensive and memory-bound at large n, the two methods are swept over different ranges: the dense Gaussian process regression is run up to $n = 20{,}000$, while the state-space method, being $\mathcal{O}(n)$ in both time and memory, is continued to $n = 100{,}000$. At each n both methods are given the same freshly drawn dataset.

For the Gaussian process regression we use the Python package scikit-learn, specifically its `GaussianProcessRegressor` with a Matérn-3/2 kernel. This is exactly the $p = 1$ case of Section 6.2, whose state-space model has dimension $m = 2$, so the two methods use the same prior. For the state-space representation we implement our own Kalman filter and RTS smoother, built from the closed-form discretization of Section 6.2 and using the remark of Section 5.1 (the stationarity identity) to form the process-noise covariance $\mathbf{Q}$. We deliberately avoid a library routine for the matrix exponential: applying a general-purpose `expm` at every step is considerably slower and would inflate the measured

[1] All code and the notebook `Experiments.ipynb` reproducing the figures of this chapter are available at `https://github.com/andreasalexandrou21/scalable-gp-state-space` (MIT license). The repository contains both the self-contained notebook and a modular implementation in `code/`; together they reproduce every figure as well as the machine-precision equivalence and linear-scaling experiments reported below.

runtime, whereas the closed form of Section 6.2 gives the exact transition matrix at negligible cost.

The kernel hyperparameters are fitted once, by the dense GP through marginal-likelihood maximization, starting from the initial guess $\sigma^2 = 0.5$, $\ell = 1$ and converging to $\sigma^2 = 0.78$, $\ell = 1.06$. The same fit can also be obtained in linear time from the Kalman marginal likelihood of Section 5.7. Section 7.2 confirms that this linear route recovers the same hyperparameters. For the timing experiments we use the single dense fit throughout, so that both methods share one fixed prior. Refitting at each n is unnecessary for a timing comparison: the per-step cost of both methods is fixed by the model order, not by the hyperparameter values, so reusing the same prior only removes a source of measurement noise. Sharing an identical prior also means that any difference between the two posteriors is purely numerical rather than a modeling difference. For the timing protocol we report wall-clock time, taking the fastest of ten repeats, measured separately for each method at each n.

7.2 Posterior Equivalence

We now verify that the state-space posterior is the exact Gaussian process posterior, not an approximation to it. By construction (Chapters 4 to 6) the two models share the same prior, so their full posteriors are guaranteed to be identical. This experiment checks that identity numerically. We take $n = 50$ observations, deliberately small so that the $\mathcal{O}(n^3)$ dense solve is a cheap and reliable ground truth, and both methods use the hyperparameters fitted in Section 7.1. We evaluate both posteriors on a dense grid of 400 points over $[0, 10]$ and compare the posterior mean and standard deviation pointwise. Each test point is inserted into the sorted sequence of times with no measurement update, so the filtering and smoothing recursions return the posterior at that time conditioned on all observations. The results are

$$\max |\mu_{\mathcal{GP}} - \mu_{\mathrm{SS}}| = 5.19 \times 10^{-15} \qquad \text{and} \qquad \max |\sigma_{\mathcal{GP}} - \sigma_{\mathrm{SS}}| = 3.22 \times 10^{-15}.$$

Both are at the level of double-precision round-off (machine epsilon is about 2.2×10^{-16}), so the recursions reproduce the dense Gaussian process posterior as exactly as floating-point arithmetic allows. We compare the marginal mean and variance here; the equality of the full joint posterior is guaranteed by the construction. This agreement is exact at $n = 50$ and is not an approximation that improves with n, in contrast to the sparse and variational methods mentioned in the introduction.

The equivalence extends to hyperparameter selection. When the hyperparameters are fitted by maximizing the marginal likelihood, computed densely for the Gaussian process and through the Kalman forward pass of Section 5.7 for the state-space model, the two routes recover the same optimum: both return $\sigma^2 = 0.78$ and $\ell = 1.06$, agreeing to within 10^{-7}. This empirically confirms the identity of Section 5.7: the filter rebuilds the exact dense marginal likelihood for free, from innovations it already computes, so the cheap linear-time objective and the expensive dense one are maximized at the same hyperparameters.

Figure 7.1 shows the two posteriors side by side, and they are visually indistinguishable.

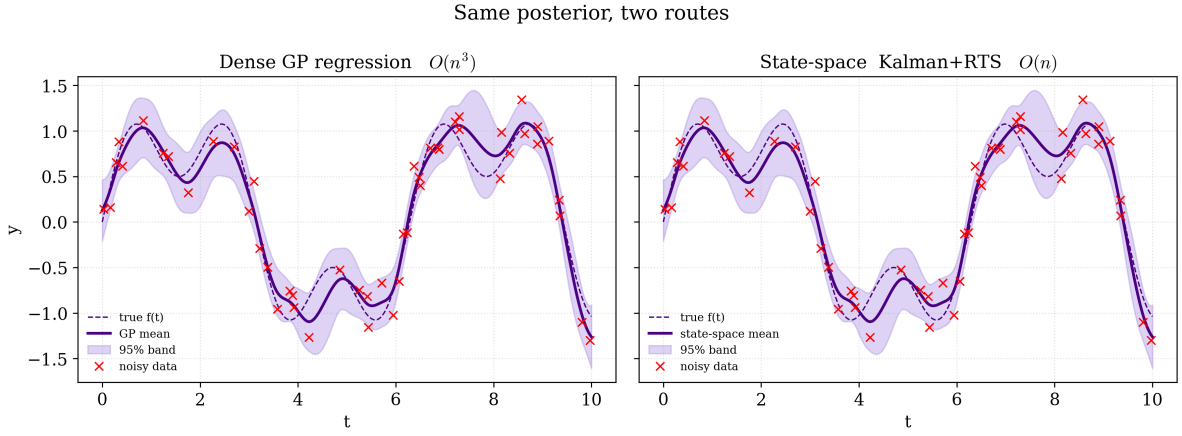


Figure 7.1: Dense Gaussian process posterior (left) and state-space Kalman–RTS posterior (right) on identical data, for a Mat'ern $\nu = 3/2$ prior with length-scale $\ell = 1.06$, signal variance $\sigma^2 = 0.78$, and noise variance $\sigma^2_{\text{noise}} = 0.04$. The two are visually indistinguishable.

7.3 Computational Cost and Scaling

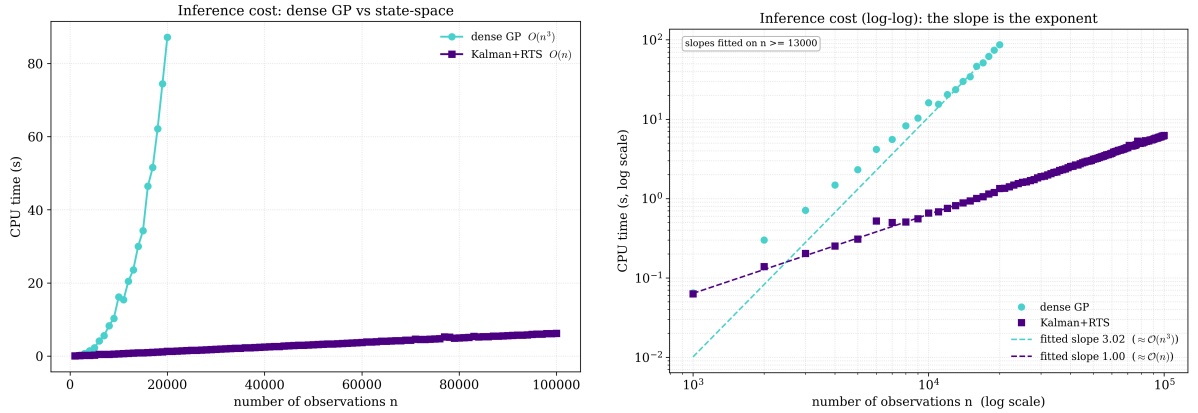
In this section we demonstrate the $\mathcal{O}(n^3)$ cost of Gaussian process regression and the corresponding $\mathcal{O}(n)$ cost of the state-space formulation. We sweep the number of observations n in steps of 1000 and, for each n, measure the time each method takes to compute the posterior. For the dense method we time the Cholesky factorization of $\mathbf{K} + \sigma^2_{\text{noise}}\mathbf{I}_n$, which is the $\mathcal{O}(n^3)$ bottleneck that any exact dense posterior must pay; for the state-space method we time the full forward filter and backward smoother, which together produce the posterior. As in Section 7.1 the hyperparameters are held fixed, so the timings reflect inference cost alone, and each measurement is the fastest of ten repeated runs. Reporting the minimum is appropriate because timing noise is one-sided: background activity can only inflate a measurement above the true compute time, never reduce it below, so the fastest run is the least contaminated estimate.

We read the asymptotic costs from a log-log plot, on which a power law $t = c\,n^p$ is a straight line of slope p. Fitting the slope on the tail $n \geq 13{,}000$, where both methods have entered their asymptotic regime, gives 3.02 for the dense regression and 1.00 for the state-space method, in agreement with the predicted 3 and 1. We fit only on the tail because at smaller n the runtime is dominated not by the leading term but by fixed overheads that flatten the apparent slope.

These overheads have two main sources. The dense solver is a single call into highly optimized, multithreaded, compiled linear algebra (the LAPACK Cholesky factorization), so its small-n cost is tiny and dominated by setup. The state-space method, by contrast, is a Python-level loop over the n observations, and each step carries a fixed

cost (the interpreter, function calls, and small array allocations) that is large relative to the 2×2 arithmetic it performs. These same overheads also make the dense solver faster in absolute terms at small n: the curves cross between $n = 1000$ and $n = 2000$. This crossover is implementation-dependent, and a compiled state-space loop would move it to smaller n.

The practical effect is large. At the largest size at which both methods were run, $n = 20{,}000$, the dense regression takes about 87 seconds, whereas the state-space method takes under 1.5 seconds, a speed-up of roughly $65\times$. We cap the dense sweep at $n = 20{,}000$, because storing the $n \times n$ covariance matrix alone costs $\mathcal{O}(n^2)$ memory and becomes impractical on the test machine. The state-space method, remaining $\mathcal{O}(n)$ in both time and memory, continues to $n = 100{,}000$, where it still completes in about 6 seconds. Figure 7.2 shows the runtimes: the linear axes in Figure 7.2a make the crossover and the cubic blow-up visible, while the log-log axes in Figure 7.2b show the fitted slopes.



(a) Linear axes.

(b) Log-log axes, with fitted tail slopes.

Figure 7.2: Inference cost against the number of observations n, for dense Gaussian process regression and the state-space method. (a) Linear axes show the crossover between $n = 1000$ and $n = 2000$ and the cubic blow-up of the dense method against the flat state-space cost; the dense sweep stops at $n = 20{,}000$ while the state-space method continues to $n = 100{,}000$. (b) Log-log axes, where the fitted tail slopes (dense 3.02, state-space 1.00) match the predicted 3 and 1.

8 Conclusion

Gaussian process regression is a flexible approach to nonparametric Bayesian regression. It places a prior directly over functions, returns a full posterior, and quantifies its own uncertainty without committing to a fixed parametric form. Its central drawback is computational: inference requires inverting the dense $n \times n$ covariance matrix, which costs $\mathcal{O}(n^3)$ in time and $\mathcal{O}(n^2)$ in memory. This quickly becomes infeasible as the number of observations grows. For a long time the response to this problem was either to make an approximation, or to restrict the model to a covariance matrix with special structure that is cheap to invert. This is what motivated this thesis: can we reduce the computational complexity of Gaussian process regression, for a stationary kernel on temporal data, from cubic to linear?

In this thesis we have shown that this is possible for specific kernels, by means of a state-space representation. We showed, step by step, that a stationary kernel whose spectral density is rational in ω^2 can be represented as the output of a linear time-invariant stochastic differential equation. This identification turns the Gaussian process into a linear-Gaussian state-space model, in which the exact posterior is produced by the Kalman filter on a forward pass and the Rauch–Tung–Striebel smoother on a backward pass, at a cost linear in the number of observations. The construction rests on the following chain of results. We first determine the spectral density of the output of a linear system (Lemma 4.1), and obtain its stationary covariance from the steady-state Lyapunov equation (Proposition 4.2). We then show that the spectral density of the output of an LTI SDE is rational (Lemma 4.3), which sets the stage for the converse direction: given a stationary covariance kernel, a well-known result from the engineering literature lets us construct an LTI SDE whose output has exactly that covariance, by spectral factorization (Proposition 4.4). The Matérn family is a concrete instance of such stationary kernels: we showed that the choice $\nu = p + 1/2$ yields a rational spectral density and a state of dimension $m = p + 1$, and we carried the case $p = 1$, the Matérn-3/2 covariance with $m = 2$, all the way through to an explicit model and to all of our experiments.

We confirmed all of the above claims experimentally. We first showed that the state-space posterior matches the dense Gaussian process posterior to machine precision, with a maximum discrepancy of order 10^{-15} in both the mean and the standard deviation, which means that it produces the exact same posterior and not an approximation. We then showed that the cost does indeed improve from cubic to linear: we timed how long each method takes to compute the posterior for a range of numbers of observations, and on log-log axes the dense regression has a fitted slope of about three while the state-space representation has a slope of about one, confirming exactly what we found in theory. Finally, the same equivalence extends to hyperparameter selection: fitting the kernel

hyperparameters by maximizing the marginal likelihood, computed densely in one case and assembled from the Kalman filter's innovations in the other, returns the same values from both routes, so the linear-time objective of Section 5.7 can replace the dense one without changing the result.

The formulation has also been put to use in practice. Applications include Gaussian process regression run in real time on a smartphone [19], the mapping of urban air pollution across a city sensor network [6], and the estimation of robot trajectories in continuous-time simultaneous localization and mapping [2].

The scope does, however, have boundaries. The construction applies to stationary temporal processes whose spectral density is rational. The Matérn family fits this requirement exactly, but many kernels do not. The squared-exponential kernel is one example: it does not admit an exact representation of this form, and although its spectral density can be approximated by a rational function (for instance through a Taylor expansion), this returns us to an approximation rather than an exact solution, which is the very thing we set out to avoid. A second point to keep in mind is the dimension of the state. The per-step cost grows cubically in this dimension, so even though it stays constant in the number of observations, a kernel that requires a high state dimension raises the overall cost considerably. Finally, the formulation developed here is for temporal data; a spatio-temporal version exists, but it is not covered by the construction given here.

Looking back on the project, the work began by reading widely in two largely separate literatures, the machine-learning treatment of Gaussian processes and the engineering treatment of state-space models and Kalman filtering, and then finding the bridge between them. The main difficulty worth mentioning was the engineering notation and conventions, which differ considerably from those used in mathematics. One of the contributions of this thesis is precisely this translation: we have recast the state-space and stochastic-differential-equation machinery, usually presented in an engineering style, into a self-contained mathematical structure aimed at mathematics students. Natural directions for future work follow from the limitations above: extending the construction to a wider class of kernels, to non-Gaussian likelihoods, and to the spatio-temporal setting. What remains is a single, clean message: for stationary Gaussian processes in time, the cubic barrier is not fundamental, and the exact posterior is available at linear cost.

Bibliography

[1] Sivaram Ambikasaran, Daniel Foreman-Mackey, Leslie Greengard, David W. Hogg, and Michael O'Neil. Fast direct methods for Gaussian processes. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 38(2):252–265, 2016.

[2] Sean Anderson and Timothy D. Barfoot. Full STEAM ahead: Exactly sparse Gaussian process regression for batch continuous-time trajectory estimation on SE(3). In *2015 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 157–164. IEEE, 2015.

[3] Rick Durrett. *Probability: Theory and Examples*. Cambridge Series in Statistical and Probabilistic Mathematics. Cambridge University Press, Cambridge, 5 edition, 2019.

[4] Gene H. Golub and Charles F. Van Loan. *Matrix Computations*. Johns Hopkins University Press, Baltimore, MD, 4 edition, 2013.

[5] Jochen Görtler, Rebecca Kehlbeck, and Oliver Deussen. A visual exploration of Gaussian processes. *Distill*, 4(4):e17, 2019.

[6] Oliver Hamelijnck, William J. Wilkinson, Niki A. Loppi, Arno Solin, and Theodoros Damoulas. Spatio-temporal variational Gaussian processes. *Advances in Neural Information Processing Systems*, 34:23621–23633, 2021.

[7] Jouni Hartikainen and Simo Särkkä. Kalman filtering and smoothing solutions to temporal Gaussian process regression models. In *2010 IEEE International Workshop on Machine Learning for Signal Processing*, pages 379–384. IEEE, 2010.

[8] Rudolph Emil Kalman. A new approach to linear filtering and prediction problems. *Transactions of the ASME–Journal of Basic Engineering*, 82(1):35–45, 1960.

[9] Fima C. Klebaner. *Introduction to Stochastic Calculus with Applications*. Imperial College Press, London, 3 edition, 2012.

[10] Armin Küper and Steffen Waldherr. Numerical gaussian process kalman filtering for spatiotemporal systems. *IEEE Transactions on Automatic Control*, 68(5):3131–3138, 2022.

[11] Dan Margalit and Joseph Rabinoff. Interactive linear algebra. `https://textbooks.math.gatech.edu/ila/`, 2019. Georgia Institute of Technology.

[12] Bertil Matérn. Spatial variation: Stochastic models and their application to some problems in forest surveys and other sampling investigations. *Meddelanden från Statens Skogsforskningsinstitut*, 49(5), 1960.

[13] Joaquin Quiñonero-Candela and Carl Edward Rasmussen. A unifying view of sparse approximate Gaussian process regression. *Journal of Machine Learning Research*, 6:1939–1959, 2005.

[14] Carl Edward Rasmussen and Christopher K. I. Williams. *Gaussian Processes for Machine Learning*. MIT Press, Cambridge, MA, 2006.

[15] Herbert E. Rauch, F. Tung, and Charlotte T. Striebel. Maximum likelihood estimates of linear dynamic systems. *AIAA Journal*, 3(8):1445–1450, 1965.

[16] Simo Särkkä and Arno Solin. *Applied Stochastic Differential Equations*. Cambridge University Press, Cambridge, 2019.

[17] Simo Särkkä and Lennart Svensson. *Bayesian Filtering and Smoothing*. Cambridge University Press, Cambridge, 2 edition, 2023.

[18] Arno Solin. *Stochastic Differential Equation Methods for Spatio-Temporal Gaussian Process Regression*. PhD thesis, Aalto University, Espoo, Finland, 2016.

[19] Arno Solin, James Hensman, and Richard E. Turner. Infinite-horizon Gaussian processes. *Advances in Neural Information Processing Systems*, 31:3486–3495, 2018.

[20] Michalis K. Titsias. Variational learning of inducing variables in sparse Gaussian processes. In *Proceedings of the 12th International Conference on Artificial Intelligence and Statistics*, pages 567–574. PMLR, 2009.

[21] Andrew Gordon Wilson and Hannes Nickisch. Kernel interpolation for scalable structured Gaussian processes (KISS-GP). In *Proceedings of the 32nd International Conference on Machine Learning*, pages 1775–1784. PMLR, 2015.